



DEPARTMENT OF ELECTRICAL
AND COMPUTER ENGINEERING

MIGUEL CHEN

BSc in Electrical and Computer Engineering

ADVERSARIAL ATTACKS ON DRL BASED NETWORK ROUTING SYSTEMS

MASTER'S THESIS

MASTER IN ELECTRICAL AND COMPUTER ENGINEERING
SPECIALIZATION IN TELECOMMUNICATIONS

NOVA University Lisbon
January, 2026



DEPARTMENT OF ELECTRICAL
AND COMPUTER ENGINEERING

ADVERSARIAL ATTACKS ON DRL BASED NETWORK ROUTING SYSTEMS

MASTER'S THESIS

MIGUEL CHEN

BSc in Electrical and Computer Engineering

Supervisor: Pedro Miguel Figueiredo Amaral
Full Professor, NOVA University Lisbon

Examination Committee

MASTER IN ELECTRICAL AND COMPUTER ENGINEERING
SPECIALIZATION IN TELECOMMUNICATIONS

NOVA University Lisbon
January, 2026

Adversarial Attacks On DRL Based Network Routing Systems

Master's Thesis

Copyright © Miguel Chen, NOVA School of School and Technology, NOVA University Lisbon.

The NOVA School of School and Technology and the NOVA University Lisbon have the right, perpetual and without geographical limitations, to archive and publish this dissertation in print or digital form, or by any other existing or future means, and to disseminate it through scientific repositories and to allow its copying and distribution for non-commercial educational or research purposes, provided that proper credit is given to the author and the publisher.

This document was produced with the pdfL^AT_EX [1] typesetting system and the NOVAthesis template (v7.9.1) [2].

- [1] The L^AT_EX Project team. *L^AT_EX - A document preparation system*. 1985. URL: <https://www.latex-project.org> (visited on 2025-12-29).
- [2] J. M. Lourenço. *The NOVAthesis L^AT_EX Template User's Manual*. NOVA University Lisbon. 2021. URL: <https://github.com/joaomlourengo/novathesis/raw/main/template.pdf>.

DISCLOSURE OF DELEGATION TO GENERATIVE ARTIFICIAL INTELLIGENCE

Miguel Chen declares¹ the use of Generative AI in the research work and writing process of this document. According to the GAIDeT taxonomy [3], the following tasks were delegated to Generative AI tools under full human supervision:

Conceptualization

- | | | |
|--|--|---|
| ☐ Idea generation | ☐ Formulating research questions and hypotheses | ☐ Feasibility assessment and risk evaluation |
| ☐ Defining the research objective | | ☐ Preliminary hypothesis testing |

Literature Review

- | | | |
|---|--|---|
| ☒ Literature search and systematization | ☒ Writing the literature review | ☐ Evaluation of novelty and identification of gaps |
| | ☐ Analysis of market trends and/or patent environment | |

Methodology

- | | | |
|--|--|--|
| ☐ Research design | ☐ Development of experimental or research protocols | ☐ Selection of research methods |
|--|--|--|

Software Development and Automation

- | | | |
|---|---|--|
| ☐ Code generation | ☐ Process automation | ☒ Creation of algorithms for data analysis |
| ☒ Code optimization | | |

Data Management

- | | | |
|--|---|--|
| ☐ Data collection | ☐ Data curation and organization | ☐ Reproducibility testing |
| ☐ Validation | ☐ Data analysis | |
| ☐ Data cleaning | ☐ Visualization | |

¹This declaration was generated using pdfL^AT_EX [1] and the L^AT_EX package aidisclose [2].

Writing and Editing

- | | | |
|--|--|--|
| ☒ Text generation | ☐ Formulation of conclusions | ☐ Translation |
| ☒ Proofreading and editing | ☐ Adapting and adjusting emotional tone | ☐ Reformatting |
| ☒ Summarizing text | | ☐ Press releases and outreach materials |

Ethics Review

- | | | |
|--|---|--|
| ☐ Bias analysis and discrimination assessment | ☐ Ethical risk analysis | ☐ Data confidentiality monitoring |
| | ☐ Monitoring compliance with ethical standards | |

Supervision

- | | | |
|---|--|--|
| ☐ Quality assessment | ☐ Identification of limitations | ☐ Publication support |
| ☐ Trend identification | ☐ Recommendations | |

Generative AI tools used: Claude Opus 5.

CONTENTS

Disclosure of Delegation to Generative Artificial Intelligence	iii
Contents	v
List of Figures	ix
List of Tables	xi
List of Algorithms	xiii
1 Introduction	1
1.1 Context and Motivation	1
1.2 Problem Statement	2
1.3 Approach	2
1.4 Contributions	3
1.5 Document Structure	4
2 Literature review	5
2.1 DRL definition and overview	5
2.2 Deep Q-Networks	6
2.2.1 Duelling Q-networks	7
2.3 MADDPG	8
2.3.1 MADDPG in Network Routing	9
2.4 Taxonomy on Adversarial attacks on MADDPG	10
2.4.1 State-Adversarial Markov Decision Processes	10
2.4.2 Observation Space Attacks	12
2.4.3 Action-Space Attacks	13
2.4.4 Communication-Space Attacks	14
2.4.5 Poisoning and Backdoor Attacks	14
2.4.6 Transferability and Black-Box Attacks	15
2.4.7 FGSM	15
2.4.8 PGD	16
2.4.9 Overview of adversarial attacks	18

2.5	Graph Neural Networks	19
2.5.1	GNN applications in network routing	21
2.6	Taxonomy on Adversarial attacks on GNNs	22
2.6.1	Attacker knowledge	22
2.6.2	Attack Timing	23
2.6.3	Attack Strategy (Perturbation Goal)	23
2.6.4	Attack Manipulation	23
2.6.5	Attack Specificity	24
2.6.6	Task level	24
2.6.7	Examples and characteristics of adversarial attacks on GNNs	24
2.6.8	Transferability-Focused Attacks	25
3	Methodology	27
3.1	Starting Point and Scope of This Work	27
3.1.1	The routing system under test	27
3.1.2	The FGSM baseline	28
3.1.3	The learned-adversary scaffold	28
3.1.4	Contribution of this thesis	28
3.2	Victim Architectures Under Test	29
3.2.1	Central (CC) versus local (LC) critic	29
3.2.2	Simple versus duelling value head	30
3.2.3	With and without GNN encoding	30
3.2.4	Variant matrix	31
3.3	Coordinated and Strategically-Timed Attacks on MADDPG Routing Agents	31
3.3.1	Threat model and admissible perturbation set	32
3.3.2	(A) Coordinated multi-agent perturbation	32
3.3.3	(B) Strategically-timed attacks under an L_0 budget	35
3.3.4	Learned observation-space adversary	38
3.3.5	Evaluation protocol	42
4	Statistics and Results	43
4.1	Reported Statistic and Experimental Protocol	44
4.2	Results Under the FGSM-Parity Domain Clamp	44
4.2.1	Sensitivity to the timing budget	45
4.2.2	Decision flips versus delivered damage	47
4.2.3	What triggered the attacks	47
4.3	Results Under the Full Domain Clamp	49
4.3.1	Independent scope: sensitivity to the timing budget	49
4.3.2	Independent scope: decision flips versus delivered damage	51
4.3.3	Coordinated scope	52
4.3.4	Effect of coordination	53

4.4 Effect of the Domain Clamp	54
Bibliography	57

LIST OF FIGURES

2.1	Diagram of Adversarial Attack characteristics on Graph Neural Networks .	22
4.1	Independent scope, FGSM-parity domain clamp. Adversarial gap for all seven victim variants at $\varepsilon = 0.30$, $n = 30$ paired episodes. Left: gap against the requested timing budget, with the mean realised attack rate printed beneath each tick and the ungated run included as the 100% reference; filled markers denote a 95% confidence interval excluding zero, hollow markers a null. Right: the same gaps plotted against the fraction of routing decisions the attack actually changed, on a logarithmic axis, with each variant drawn as one path walking from $b = 0.10$ through to the ungated run. A thin line joins a label to its series where the label had to be moved clear of the data.	45
4.2	Independent scope, FGSM-parity domain clamp: why the timing gate fired in the attack arm ($\varepsilon = 0.30$, $n = 30$ episodes). One panel per trigger that occurred in at least one run, combining the case of Equation (3.15) that admitted the step with the congestion band of its score. Rows are victim variants and columns the requested timing budget. Each cell gives the trigger's share of that run's attacked steps, with the count beneath it, so the three panels of one cell sum to 100%. Ungated runs have no gate and are omitted.	48
4.3	Independent scope, full domain clamp. Adversarial gap for all seven victim variants at $\varepsilon = 0.30$, $n = 30$ paired episodes. Left: gap against the requested timing budget, with the mean realised attack rate printed beneath each tick and the ungated run included as the 100% reference; filled markers denote a 95% confidence interval excluding zero, hollow markers a null. Right: the same gaps plotted against the fraction of routing decisions the attack actually changed, on a logarithmic axis, with each variant drawn as one path walking from $b = 0.10$ through to the ungated run.	49

4.4	Coordinated scope, full domain clamp. Adversarial gap for all seven victim variants at $\varepsilon = 0.30$, $n = 30$ paired episodes, with the perturbation emitted jointly for the whole compromised group. Panels and markers as in Figure 4.3. A thin line joins a label to its series where the label had to be moved clear of the data.	52
-----	--	----

LIST OF TABLES

2.1	Taxonomy of Adversarial Attacks on MADDPG	18
2.2	Taxonomy of Adversarial Attacks on MADDPG part 2	19
2.3	Attacker’s Knowledge (Information Access Level)	23
2.4	Summary of Adversarial Attacks on GNNs	24
3.1	MADDPG victim variants under test, named <critic domain>-<value head>[-GNN].	31
4.1	Independent scope, FGSM-parity domain clamp. Adversarial gap in percentage points by victim variant and requested timing budget, at $\varepsilon = 0.30$ and $n = 30$ paired episodes. Bold entries have a 95% confidence interval excluding zero. Positive values indicate the attack cost the victim more than the random control of the same magnitude; negative values indicate the reverse.	46
4.2	Independent scope, FGSM-parity domain clamp. Ungated results at $\varepsilon = 0.30$, $n = 30$ paired episodes: adversarial gap with its 95% confidence interval, and the fraction of routing decisions changed by the attack and by the random control.	47
4.3	Independent scope, full domain clamp. Adversarial gap in percentage points by victim variant and requested timing budget, at $\varepsilon = 0.30$ and $n = 30$ paired episodes. Bold entries have a 95% confidence interval excluding zero. Positive values indicate the attack cost the victim more than the random control of the same magnitude; negative values indicate the reverse.	50
4.4	Independent scope, full domain clamp. Ungated results at $\varepsilon = 0.30$, $n = 30$ paired episodes: adversarial gap with its 95% confidence interval, and the fraction of routing decisions changed by the attack and by the random control.	51
4.5	Coordinated scope, full domain clamp. Adversarial gap in percentage points by victim variant and requested timing budget, at $\varepsilon = 0.30$ and $n = 30$ paired episodes. Bold entries have a 95% confidence interval excluding zero.	53

4.6	Effect of coordination under the full domain clamp: coordinated minus independent adversarial gap, in percentage points, by victim variant and requested timing budget ($\varepsilon = 0.30$, $n = 30$ paired episodes). Positive values mean coordination increased the damage. Bold entries have a 95% confidence interval on the paired difference of Equation (4.1) excluding zero.	53
4.7	Effect of the domain clamp in the independent scope: FGSM-parity minus full clamp adversarial gap, in percentage points, by victim variant and requested timing budget ($\varepsilon = 0.30$, $n = 30$ paired episodes). Positive values mean the parity-clamp adversary did more damage. Bold entries have a 95% confidence interval on the paired difference of Equation (4.2) excluding zero.	55

LIST OF ALGORITHMS

1	Coordinated multi-agent perturbation by projected gradient ascent (reference formulation, one time step)	35
2	Critical-state timing gate with rationed ties	37
3	Training the learned observation-space adversary (independent scope) . .	40

INTRODUCTION

1.1 Context and Motivation

As telecommunication networks grow in scale and heterogeneity, routing, quality of service and resource management become increasingly difficult to handle with hand-crafted rules. Traditional routing protocols such as OSPF and BGP rely on static, rule-based decisions that adapt poorly to variable traffic, heterogeneous Quality-of-Service (QoS) requirements and competing optimisation constraints. Deep Reinforcement Learning (DRL) has emerged as an alternative, allowing routing agents to learn adaptive policies directly from observations of the network rather than from explicitly programmed rules [4].

Multi-agent formulations extend this to distributed control, in which every forwarding node is an agent that decides locally while the paths chosen collectively determine end-to-end delivery. Multi-Agent Deep Deterministic Policy Gradient (MADDPG) is a natural fit for this setting: under centralised training with decentralised execution, each agent learns from a critic that may observe the whole system during training, yet acts on its own local observation once deployed [5]. Combined with Graph Neural Networks, which exploit the graph structure of the topology, such controllers have been reported to improve scalability, topology awareness and generalisation across network configurations [6–11].

The same property that makes these controllers adaptive also exposes them. A learned routing policy decides from the network state it perceives, and neural policies are known to be sensitive to small, deliberately chosen perturbations of their inputs [12, 13]. Attacks such as the Fast Gradient Sign Method (FGSM) [13], Projected Gradient Descent (PGD) [14] and the Carlini and Wagner attack [15] were developed largely for image classification, where one perturbation targets one prediction. In reinforcement learning the stakes differ: a perturbed observation changes an action, the action changes the states that follow, and the damage accumulates over a trajectory [16]. For a routing controller, an adversary who can corrupt the telemetry an agent receives, without ever touching the network itself, can redirect traffic. How vulnerable learned routing controllers are to such an adversary remains largely unexplored.

1.2 Problem Statement

This thesis continues a line of research on a MADDPG routing controller developed in earlier work [17]. A preceding dissertation in the same line attacked that controller with FGSM, perturbing each agent’s observation in a single gradient step at every time step [TODO-colleague-fgsm]. The attack altered a substantial fraction of the routing decisions taken by the victim, yet recovered only a small part of the degradation achievable in principle, because the redundancy of the topology absorbed the redirected traffic.

That result admits two readings. Either the controller is genuinely robust, or a single-step, per-agent adversary is too weak for its failure to count as evidence of robustness. The two cannot be separated without a stronger attacker, and this thesis provides one. The central research question is therefore:

Does the apparent robustness of MADDPG routing controllers to FGSM survive a stronger observation-space attack, one that coordinates its perturbation across agents and divides it across time?

It is broken down into four questions:

- RQ1** Does dividing the attack across time, spending a limited budget on the network’s most critical states, buy more damage per attacked step than perturbing every step?
- RQ2** Does coordinating the perturbation across compromised agents, rather than attacking each agent in isolation, increase the damage?
- RQ3** Which architectural choices of the victim, namely its critic domain, its value head and the use of GNN encoding, determine how vulnerable it is?
- RQ4** Is the fraction of routing decisions an attack changes a meaningful measure of the damage it does?

1.3 Approach

The attack addresses the two weaknesses of the baseline in turn, drawing on a different source of inspiration for each. Where FGSM spends its whole perturbation budget in a single step, PGD divides it into a sequence of smaller projected steps [14]. The attack carries this principle from the input into time: instead of applying a full-strength perturbation at every step, it acts on only a fraction of the steps, chosen by a critical-state score that identifies the moments at which the network’s redundancy is thinnest. Where the baseline perturbs each agent in isolation, SAJA [18] judges the perturbations of several victim agents together, by the centralised critic’s value of the joint action they induce. Inspired by it, the attack coordinates its perturbation jointly across the compromised agents, so that several of them can be steered towards the same shared link.

The attack is realised as a learned adversary. Because the victim is frozen, its policies are fixed, and against a fixed policy the problem of finding the optimal observation adversary is itself an ordinary Markov decision process [16]. The adversary is therefore trained by reinforcement learning to act in this coordinated and strategically timed way, learning which perturbation to apply from the damage its perturbations cause. Unlike FGSM, it never needs the gradients of the victim, only the ability to query its actions and a simulator to train against.

The attack is evaluated against seven victim variants that differ in their critic domain, their value head and the use of GNN encoding. To keep the comparison with the FGSM study meaningful, the attack operates on the same frozen agents and within the same ℓ_∞ perturbation budget, and is scored under the same evaluation protocol. It is evaluated under the FGSM study’s domain constraint and, as a second condition, under a stricter constraint that keeps every observation feature within $[0, 1]$, with a separately trained adversary for each. Every configuration is run as three paired arms on identical traffic: a clean arm, a random control that perturbs by the same amount in a random direction, and the attack. The damage attributed to the attack is its margin over the random control, since any perturbation of that size disturbs the controller whether or not its direction was chosen adversarially.

1.4 Contributions

The main contributions of this thesis are:

- An observation-space attack on MADDPG routing controllers that coordinates its perturbation across agents, inspired by SAJA, and divides it across time, inspired by PGD. It is realised as a learned adversary grounded in the State-Adversarial MDP.
- A critical-state timing gate that spends a limited attack budget on high-leverage moments. It includes a tie-rationing rule that makes the realised attack rate match the requested budget, which a plain threshold cannot do because the critical-state score concentrates on a few repeated values.
- A coordinated multi-agent formulation of the perturbation, motivated by showing that any attack that treats agents independently optimises a separable objective and cannot express convergence onto a shared link.
- A paired evaluation protocol with a random control, which separates the damage attributable to the attacker from that of any same-sized perturbation, and which reports decisions changed separately from delivery lost.
- An empirical study across seven victim architectures and two domain constraints. Under the FGSM study’s constraint, the independent attack substantially damages the local-critic victim with a simple value head, which the independent attack under

the stricter constraint could not harm. Under the stricter constraint, the duelling value head is instead the most vulnerable choice, and concentrating the budget on critical states buys more damage per attacked step than attacking at every step, while neither finding holds under the FGSM study’s constraint. Against one victim the trained adversary even improves delivery, so a learned adversary’s failure to harm a victim is weak evidence of robustness. Under both constraints, the fraction of decisions an attack changes does not predict the damage it does. Coordination, evaluated under the stricter constraint, significantly increases the damage done to local-critic victims without GNN encoding, while its effect on central-critic victims is mixed.

1.5 Document Structure

The remainder of this document is organised as follows. Chapter 2 reviews the relevant literature: deep reinforcement learning and its application to routing, MADDPG, adversarial attacks on reinforcement learning agents including the State-Adversarial MDP, SAJA, FGSM and PGD, and graph neural networks together with the attacks that target them. Chapter 3 describes the routing system and the FGSM baseline inherited from earlier work, the victim architectures under test, the attack with its SAJA-inspired coordination and PGD-inspired timing mechanisms, the learned adversary that realises it, and the evaluation protocol. Chapter 4 presents the results under the FGSM study’s domain constraint and under the stricter one, reporting the independent and coordinated scopes, the effect of coordination and the effect of the constraint itself. Chapter 5 concludes and outlines directions for future work.

LITERATURE REVIEW

This chapter reviews relevant literature on Multi-Agent Deep Deterministic Policy Gradient (MADDPG) and Graph Neural Networks (GNNs). We will explore their architectures, the proposed applications in network routing and current literature on their vulnerabilities to adversarial attacks.

2.1 DRL definition and overview

Deep Reinforcement Learning (DRL) is a subfield of machine learning that combines reinforcement learning (RL) principles with deep neural networks to enable agents to learn optimal policies in complex environments. In DRL, an agent interacts with an environment by taking actions based on its current state, receiving feedback in the form of rewards or penalties. The goal of the agent is to learn a policy that maximises the cumulative reward over time. Unlike traditional rule-based approaches, DRL agents learn directly from state observations, making them particularly suited for network optimisation tasks where hand-crafted rules cannot capture the full complexity of system dynamics [19–21]. In the context of network routing, DRL enables agents to discover adaptive forwarding policies that respond to real-time network conditions (congestion, link failures, traffic demands) without requiring an explicit model of the environment [22–24]. The environment is naturally modelled as a Markov Decision Process (MDP) or Partially Observable MDP (POMDP) where states encode link loads and topology information, actions correspond to next-hop or path selection decisions, and rewards capture delay, throughput, or composite QoS metrics [25].

However, DRL models are vulnerable to adversarial attacks, where malicious actors can input adversarial examples designed to mislead the model into making incorrect decisions [12], these are often called either evasion attacks when introduced during test-time or poisoning attacks when introduced during training. Lin et al. [26] further introduce ‘enchanted attacks’, which craft a sequence of adversarial perturbations to drive the agent toward specific target states [27, 28]. These attacks exploit the inherent lack of robustness in the model, leading to suboptimal decision-making in many application areas [29–31]. Common adversarial attack methods include the Fast Gradient Sign Method

(FGSM), Projected Gradient Descent (PGD), the Carlini and Wagner (CW) attack and more recently, the Distribution-Aware Projected Gradient Descent (DAPGD). [32, 33]

2.2 Deep Q-Networks

Deep Q-Networks integrate deep convolutional neural networks with Q-learning, a reinforcement learning algorithm, to learn optimal action-selection policies directly from high-dimensional sensory inputs without manual feature engineering [34, 35]. The architecture parameterises the action-value function, the Q-function, using a deep neural network whose input consists of preprocessed high-dimensional observations (e.g., raw pixel data) and whose output comprises value estimates for each possible action, representing the expected cumulative future reward associated with each action selection. Training stability, historically a critical challenge when combining neural networks with temporal-difference learning, is achieved through two key mechanisms: experience replay, a technique wherein the network is trained on minibatches of randomly sampled experiences drawn from a memory buffer, thereby breaking temporal correlations in the observation sequence and smoothing shifts in the data distribution [34, 35]. The network is trained via stochastic gradient descent to minimise the temporal-difference error, defined by the squared Bellman residual between the predicted Q-value and the target value computed as the immediate reward plus the discounted maximum Q-value of the subsequent state as estimated by the target network [34].

$$L_i(\theta_i) = \mathbb{E}_{s,a \sim \rho(\cdot)} [(y_i - Q(s, a; \theta_i))^2] \quad (2.1)$$

where the target value is:

$$y_i = \mathbb{E}_{s' \sim \mathcal{E}} \left[r + \gamma \max_{a'} Q(s', a'; \theta_{i-1}) \middle| s, a \right] \quad (2.2)$$

- $L_i(\theta_i)$: Loss function at iteration i with network parameters θ_i
- θ_i : Network parameters (weights and biases) at iteration i
- θ_{i-1} : Network parameters from the previous iteration $i - 1$ (held fixed during optimisation of L_i)
- $\mathbb{E}_{s,a \sim \rho(\cdot)}$: Expectation over state-action pairs (s, a) sampled from the behaviour distribution $\rho(s, a)$
- y_i : Target value (Bellman target) for iteration i
- $Q(s, a; \theta_i)$: Predicted action-value function (Q-function) that estimates the expected cumulative discounted reward for taking action a in state s using network parameters θ_i

- r : Immediate scalar reward received from the environment after taking action a in state s
- γ : Discount factor; a scalar in $[0, 1]$ that weights the importance of future rewards relative to immediate rewards
- $\max_{a'} Q(s', a'; \theta_{i-1})$: Maximum Q-value across all possible actions a' in the successor state s' , computed using the previous iteration's parameters θ_{i-1}
- s : Current state, represented as a sequence of preprocessed observations (e.g., stacked pixel frames)
- a : Action taken in state s ; chosen from the discrete action set
- s' : Successor state resulting from taking action a in state s
- $\mathcal{E}$: The environment (in the DQN paper: the Atari 2600 emulator [34]); transition dynamics are unknown and learned implicitly
- $\rho(s, a)$: Behaviour distribution; the probability distribution of state-action pairs encountered during training (typically from an ε -greedy policy)

[34]

DQN has been successfully applied to network routing tasks, where the agent learns to make optimal forwarding decisions based on various real-time network state observations. Geng and Zhang (2023) [36] propose a DQN-based routing protocol that optimises both path selection and energy efficiency thereby significantly extending network lifetime of underwater acoustic sensor networks (UASNs). DQL has also been proposed as a solution to help Software Defined Networks (SDN) based data-centers dealing with "mice" flows, which are short-lived flows with relatively small data sizes that can congest the network if not handled properly and "elephant" flows, that are long lived and carry large amounts of data. Two DQNs are built, one for each type of flow, that train Convolutional Neural Networks (CNN) to optimise routing decisions [37].

2.2.1 Duelling Q-networks

Dueling Network Architectures for Deep Reinforcement Learning represent a structural innovation to the standard Deep Q-Network architecture in which the action-value function is decomposed into two independent streams that share a common feature representation before final aggregation [38, 39]. Specifically, the architecture bifurcates into a value stream $V(s; \theta_v)$ that estimates the scalar value of a given state independent of action selection, and an advantage stream $A(s, a; \theta_a)$ that computes the relative advantage of executing a particular action within that state, with both streams processing shared convolutional features derived from the input observation [38]. The final action-value estimate is obtained through a duelling layer that performs a learned aggregation of these two quantities

and provides a normalisation mechanism that stabilises gradient-based learning and enforces identifiability [38]. This architectural separation facilitates more efficient learning in domains where numerous actions yield comparatively similar expected returns, by enabling the value and advantage streams to specialise in capturing specific aspects of the policy structure while maintaining a shared feature space, thereby improving sample efficiency and convergence properties [38].

In a recent paper by Zhang et al. (2022) [40] a deep duelling Q-network based routing protocol helps optimise Routing and Spectrum Allocation (RSA) in Elastic Optical Networks (EONs). The proposed method leverages the duelling architecture to effectively learn efficient resource allocation policies under dynamic traffic conditions, where connection requests arrive and terminate unpredictably. The decomposition of the Q-function into value and advantage streams allows the model to better judge the relative merits of different routing actions based on the overall network congestion and fragmentation state thereby circumventing the learning inefficiency that arises when traditional DQN attempts to differentiate between numerous similar-valued routing actions across diverse network configurations [40].

2.3 MADDPG

Multi-Agent Deep Deterministic Policy Gradient (MADDPG) is a deep reinforcement learning algorithm designed for multi-agent environments that extends actor-critic methods through a centralised training with decentralised execution framework, where each agent maintains a centralised action-value function conditioned on agent actions during training while using only local observations at execution time. To relax the assumption of knowing other agent's policies, MADDPG enables each agent to maintain an approximate policy of other agents by maximising the log-probability of observed actions with entropy regularisation, allowing subject agents to learn target agents policies before adapting accordingly. For enhanced robustness in competitive settings, MADDPG trains a collection of different sub-policies per agent, randomly selecting one sub-policy per episode and maintaining separate replay buffers for each sub-policy, which maximises the ensemble objective such that agents develop diverse behavioural strategies that prevent overfitting to specific opponent policies [5].

However, this very architecture creates exploitable attack surfaces. The centralised critic's dependence on gradient computations makes MADDPG amenable to gradient-based adversarial perturbations, while the decentralised execution phase means each agent operates independently at test time, creating coordination vulnerabilities when one or more agents receive adversarial inputs [25, 41, 42].

2.3.1 MADDPG in Network Routing

A recent paper by Tian et al. (2023) [4] proposes a MADDPG-based approach for solving the network slicing problem in 5G networks, a resource allocation problem where diverse service types with heterogeneous QoS requirements may, at any point, change their resource demands. The proposed method, MADDPG-SR, treats each virtual network function (VNF) as an agent that learns to allocate resources and migrate dynamically with the end goal of maintaining QoS requirements in every slice. The results show the proposed method outperforms non-MADDPG implementations [4].

A recent paper by Huang et al. (2025) [8] proposes ERA-MADDPG as an elastic routing scheme for software-defined networks (SDNs) that casts routing optimisation as a cooperative multi-agent decision-making problem and uses Multi-Agent Deep Deterministic Policy Gradient (MADDPG) as its learning backbone. In this framework, network entities (e.g., switches or abstract routing agents) learn deterministic policies that select paths or adjust forwarding behaviour based on observed traffic and topology information, while a centralised critic exploits global state during training to stabilise learning in the inherently non-stationary multi-agent environment. The resulting actors are then deployed in a decentralised fashion, allowing each agent to make fast local decisions that collectively adapt the end-to-end routes to time-varying traffic conditions and QoS demands. By leveraging MADDPG, ERA-MADDPG overcomes key limitations of static routing and single-agent RL methods, notably poor scalability and weak coordination, enabling continuous “elastic” adjustment of routes to jointly reduce delay, packet loss, and congestion while improving load balance. This method outperforms other DRL based algorithms on training efficiency and recovery in dynamic network topologies [8].

Another recent design, RS-MADDPG [43], enables each node to learn a deterministic policy that maps local observations to continuous routing decisions (e.g., flow splitting ratios or next-hop selections) while implicitly accounting for the actions of other agents, thereby mitigating the non-stationarity that plagues independent RL approaches. By leveraging MADDPG’s capacity for continuous action spaces and multi-agent credit assignment, RS-MADDPG dynamically adjusts routes in response to real-time traffic patterns, link failures, and varying QoS demands without requiring manual reconfiguration. The algorithm’s effectiveness is further enhanced by its ability to converge approximately 30 training cycles earlier than baseline methods, demonstrating that the Centralised Training Decentralised execution (CTDE) paradigm, characteristic of MADDPG, accelerates learning of cooperative policies that jointly minimise latency, balance loads, and maximise throughput. Consequently, RS-MADDPG not only outperforms conventional protocols in key performance metrics but also provides a scalable, adaptive solution for delivering differentiated QoS services in modern SDN environments [43].

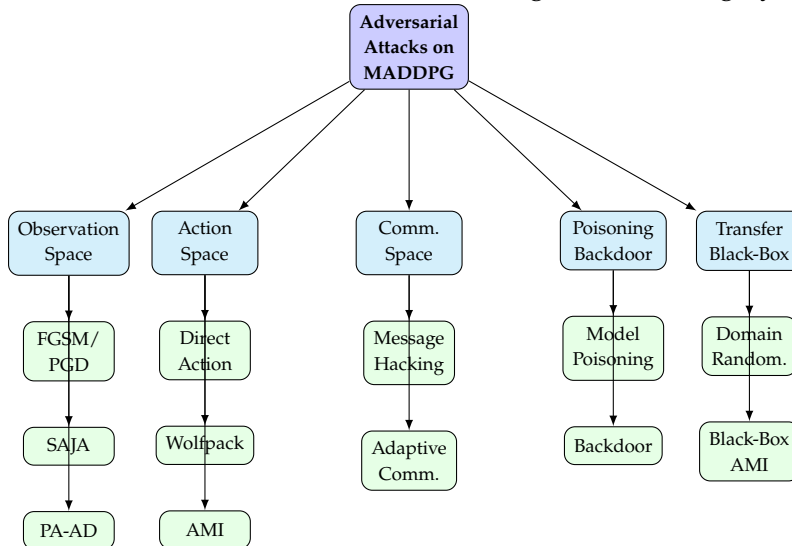
An interesting development based on the MADDPG algorithm is the Dynamic Packet Routing Algorithm using Multi-Agent Reinforcement Learning with Emergent Communication (DRAMA) [10]. DRAMA utilises the Centralised Training Decentralised Execution

(CTDE) framework of MADDPG, but redesign the value-function architecture and information flow to overcome MADDPG’s key limitations in packet routing. While MADDPG uses a Type-II critic that takes explicit joint observation, action pairs as input, requiring fixed, enumerated discrete actions and becoming brittle when the action set changes, DRAMA replaces this with a Q-network score layer that operates on topology-aware latent features extracted by an observation encoder and a graph-based emergent communication layer. This preserves the CTDE advantage of learning from joint observations and rewards while decoupling the Q-function from a fixed action index space, so the same trained network can seamlessly score newly appearing neighbours or cope with link/router failures without the need for extensive retraining. Empirically, this MADDPG-based CTDE training combined with DRAMA’s communication and Q-network design yields comparatively better delivery rates and lower end-to-end latency than regular MADDPG implementations across a range of traffic loads and under dynamic topologies, demonstrating that DRAMA leverages the strengths of MADDPG’s training paradigm while addressing its natural difficulty with scalable, adaptive routing [10].

Although the usage of MADDPG in network routing has increased in recent years, there is lack of research on the effects of adversarial attacks in this context.

2.4 Taxonomy on Adversarial attacks on MADDPG

Adversarial attacks against MADDPG can be systematically categorised along three dimensions: (1) perturbation domain (observations, actions, inter-agent communication, and training-time manipulation of rewards or the model itself), (2) attack timing (training-time vs. test-time), and (3) attacker knowledge (white-box, gray-box, black-box).



2.4.1 State-Adversarial Markov Decision Processes

Adversarial examples in supervised learning perturb a single input to change a single prediction, and their success is judged on that one output. Reinforcement learning does not

share this structure. An agent’s decision determines the next state it visits, so a perturbation that alters one action also alters the trajectory that follows, and its true cost is measured in lost cumulative return rather than in a single incorrect output. Zhang et al. [16] formalise this setting as the **State-Adversarial Markov Decision Process (SA-MDP)**, a framework for reasoning about attacks that corrupt what an agent observes.

An SA-MDP is defined by the tuple $(\mathcal{S}, \mathcal{A}, B, R, p, \gamma)$, where $\mathcal{S}$, $\mathcal{A}$, R , p and γ are the state space, action space, reward function, transition probability and discount factor of an ordinary MDP, and $B(s) \subseteq \mathcal{S}$ is the set of perturbations the adversary may apply at state s , commonly chosen as an ℓ_p -norm ball around s . The adversary is a function $v : \mathcal{S} \rightarrow \mathcal{S}$ constrained to $v(s) \in B(s)$, and is assumed to be deterministic, stationary and Markovian. The defining feature of the model is where the perturbation acts: the agent selects its action from the perturbed observation, $a \sim \pi(\cdot | v(s))$, but the environment continues to evolve from the true state s . The value of a policy π under an adversary v therefore satisfies

$$\tilde{V}_{\pi \circ v}(s) = \sum_{a \in \mathcal{A}} \pi(a | v(s)) \sum_{s' \in \mathcal{S}} p(s' | s, a) \left[R(s, a, s') + \gamma \tilde{V}_{\pi \circ v}(s') \right], \quad (2.3)$$

so the adversary corrupts only what the agent perceives, never the dynamics or the reward the agent actually receives [16].

The central result for attack design is that, for a *fixed* policy π , finding the optimal adversary is itself a standard reinforcement learning problem. Zhang et al. show (Lemma 1) that there exists an ordinary MDP whose action space is the state space itself, so that the adversary’s action is the perturbed observation it presents to the agent, and whose reward is the negative of the agent’s expected reward; the optimal policy of this MDP is precisely the optimal adversary v^* of the SA-MDP [16]. Two consequences follow. First, the optimal adversary can be learned with an ordinary reinforcement learning algorithm that treats the victim as part of the environment. Second, because that adversary maximises a reward accumulated over time, it optimises its perturbations jointly across the trajectory, accounting for how each one shapes the states it will later perturb, whereas gradient-based attacks such as FGSM and PGD select each perturbation greedily from the current step alone.

The framework also clarifies why robustness to observation attacks is hard to obtain. Zhang et al. prove (Theorem 4) that under the optimal adversary an optimal Markovian policy need not exist, so a defender cannot in general compute a policy that is simply best against the worst case. They further bound the damage an adversary can inflict (Theorem 5):

$$\max_{s \in \mathcal{S}} \{V_\pi(s) - \tilde{V}_{\pi \circ v^*}(s)\} \leq \alpha \max_{s \in \mathcal{S}} \max_{\hat{s} \in B(s)} D_{TV}(\pi(\cdot | s), \pi(\cdot | \hat{s})), \quad (2.4)$$

where α is a state-independent constant and D_{TV} denotes total variation distance. The loss in value is therefore controlled by how far the policy’s action distribution can be shifted within the perturbation set. This motivates their state-adversarial regularisers, which penalise that shift during training and are instantiated for PPO, DDPG and DQN

as SA-PPO, SA-DDPG and SA-DQN; the DDPG variant is the most closely related to the MADDPG family studied in this work [16].

The same work introduces two attacks that require no access to the victim’s critic. The **Maximal Action Difference (MAD)** attack searches for a perturbed state $\hat{s}$ that maximises $D_{KL}(\pi(\cdot | s) \parallel \pi(\cdot | \hat{s}))$, targeting directly the policy shift that Equation 2.4 identifies as the source of value loss, while the **Robust Sarsa (RS)** attack first learns an estimate of Q^π on-policy and then uses it to guide a gradient-based perturbation [16]. An action-difference objective of this kind also appears in the SAJA attack discussed in Section 2.4.2.

Building on Lemma 1, Zhang et al. [44] propose **Alternating Training with Learned Adversaries (ATLA)**. With the agent’s policy held fixed, an adversary is trained by policy gradient to solve the adversary MDP, a large negative reward discouraging perturbations outside the admissible set; with that adversary then held fixed, the agent is trained against it, and the two phases alternate. The learned adversary proves considerably stronger than the per-step attacks that preceded it, driving victim reward substantially lower than critic-based, MAD, RS and snooping attacks, and it can trick an agent into moving in the opposite direction of its goal rather than merely halting its progress [44]. Learning the adversary directly in the state space becomes costly as the state dimension grows, however, because the adversary’s action space is the full observation space; the PA-AD attack discussed in Section 2.4.2 addresses this limitation by decomposing the adversary into components that operate on policy perturbations rather than on raw states [45].

The SA-MDP framework is formulated for a single agent, and these works do not address multi-agent settings in which several agents may be perturbed at once. Its relevance to this thesis lies in Lemma 1. An adversary attacking a frozen victim faces a fixed policy, so the attack can be learned rather than recomputed by gradient ascent at every step, and the learned adversary need not differentiate the victim at all. This is the basis on which the attack developed in this work, whose coordination mechanism is inspired by SAJA and whose timing mechanism is inspired by PGD, is realised as a learned adversary.

2.4.2 Observation Space Attacks

State-space attacks perturb the agent’s observations before the actor network processes them. These attacks exploit the gradient-based learning mechanisms underlying neural network policies.

Gradient-Based State Perturbations: The foundational approach applies the Fast Gradient Sign Method (FGSM) or Projected Gradient Descent (PGD) to compute adversarial observations. FGSM performs a single-step perturbation in the gradient direction of the loss function [13], while PGD extends this through iterative optimisation with projection [14]. However, according to Guo et al. [18] standard FGSM attacks on MADDPG agents show limited effectiveness because the critic’s Q-value gradients alone provide unreliable guidance, imperfect critic training leads to noisy gradients [46].

State-Action Joint Attacks (SAJA): Guo et al. [18] observe that earlier work attacks

either the states or the actions of a multi-agent system in isolation, and propose SAJA to exploit both together. SAJA is a white-box attack on multi-agent actor-critic algorithms with a centralised critic, and it operates in two phases. In the first, the observations of the victim agents are perturbed by iterated sign-gradient ascent projected onto an ℓ_∞ ball, using both the actor and the critic: the actions a' that the policies produce from the perturbed observations are scored by the centralised critic, and the loss combines the negative critic value $-Q(s^{\text{adv}}, a')$ with an action-difference term $\|a' - a^{(0)}\|_2$. In the second phase, starting from the perturbed state, the critic is used again to craft a perturbation of the actions themselves. The action-difference term is grounded in a bound: the paper shows (Theorem 2) that the degradation of the value function is bounded by, and positively correlated with, the multi-agent action difference [18].

SAJA is evaluated on MADDPG and FACMAC in the Multi-Agent Particle Environment, with one to three victim agents. It causes reward drops of up to 9.90% on 3-agent FACMAC, bypasses the M3DDPG, PA-AD and ATLA defences, and the joint attack proves more effective than the corresponding state-only and action-only attacks [18].

What makes SAJA relevant to this thesis is how it judges a perturbation: not by its effect on one agent in isolation, but through the centralised critic’s value of the joint action that the agents take once the victims’ observations are perturbed. The coordination mechanism of the attack developed in this work takes this idea as its inspiration (Section 3.3.2): the perturbations of the compromised agents are chosen together, by their combined effect on the network. Unlike SAJA, the threat model here perturbs observations only, never actions, and the perturbation is produced by a learned adversary rather than by gradient ascent on the victim’s critic.

Policy Adversarial Actor-Director (PA-AD): Rather than directly learning state perturbations in high-dimensional spaces, PA-AD proposes a Policy Adversary MDP (PAMDP) where an optimal policy induces the optimal state adversary. The attack employs collaborative "director" and "actor" components: the director learns which policy perturbation directions are most effective, while the actor crafts state perturbations to realise those directions. PA-AD achieves state-of-the-art performance on Atari and MuJoCo tasks, and combined with adversarial training, produces robust policies that significantly outperform baseline defences [45].

2.4.3 Action-Space Attacks

Action-space attacks directly manipulate an agent’s output actions, either through direct perturbation of action values or by injecting adversarial agents that execute perturbed actions.

Direct Action Manipulation: SAJA’s action-only variant applies gradient ascent on the critic network conditioned on the perturbed state to craft final adversarial actions. These attacks are particularly effective when attackers possess partial control over one or more agent’s actuators [18].

Malicious Agent Injection: Rather than perturbing a single agent’s actions, attackers can inject fully adversarial agents into the multi-agent system. These agents learn to disrupt the victim agent’s policies through direct behavioural interference. The Wolfpack attack exemplifies this: by targeting a single agent and subsequently attacking its supporting agents, adversaries exploit the inherent interdependencies in cooperative systems. The Wolfpack approach reduces rewards by up to 51.8% and severely impacts SMAC scenarios, proving far more effective than random or single-agent attacks [47].

Adversarial Minority Influence (AMI): AMI leverages social psychology principles where a minority (adversary) unilaterally sways the majority (victims) toward worst-case failure. AMI employs two innovations: (1) a unilateral influence filter adapted from mutual information that maximises adversary-to-victim influence while excluding reverse influence, and (2) a targeted adversarial oracle that learns worst-case target actions. AMI successfully attacks real robot swarms and simulation environments including StarCraft II and multi-agent MuJoCo [42].

2.4.4 Communication-Space Attacks

Multi-agent systems with explicit communication channels face a distinct attack surface: adversarial manipulation of inter-agent messages.

Message Hijacking: The paper "Hijacking Robot Teams Through Adversarial Communication" by Wu et al. [48] demonstrates that an adversarial policy can intercept and modify messages between agents. The authors report several-fold performance degradation (e.g., more than four-fold in some simulated predator–prey tasks and roughly a factor of two in Robotarium experiments), illustrating that adversarial communication can catastrophically disrupt team performance. The attack bypasses agents’ native policies because the manipulated messages are indistinguishable from benign communication within the agents’ learned policies.

Communication-Robust Learning: Advanced attacks model adversarial communication as a cooperative problem, learning adaptive attack strategies that persistently disrupt coordination. The work on differentially private communication reveals that privacy-preserving mechanisms, while protecting against eavesdropping, do not inherently defend against adversarial message perturbations [49, 50].

2.4.5 Poisoning and Backdoor Attacks

Training-time attacks embed long-term vulnerabilities into learned policies [28].

Model Poisoning via MARL Coordination: In federated learning with multiple poisoning attackers, attackers coordinate using MADDPG itself to craft effective global model updates that degrade overall system performance. The attack assumes attackers can approximate the benign data distribution, then formulate the poisoning problem as an MDP where attackers jointly learn attack policies through MADDPG, outperforming baseline poisoning methods [51].

Backdoor Attacks (TrojDRL, BACKDOORL, BadRL, MARLNet): These frameworks embed trigger conditions during training. Once triggered, agents execute attacker-specified actions while appearing normal otherwise. MARLNet specifically targets multi-agent scenarios using imperceptible triggers and reward manipulation. Advanced backdoor frameworks achieve near-perfect attack success rates (ASR approximating 99%) with minimal utility loss on benign tasks. Backdoor Vaccination, where defenders inject controlled triggers—offers a proactive defence by leveraging the backdoor concept itself [28, 52–54].

2.4.6 Transferability and Black-Box Attacks

Domain Randomisation for Transfer: Adversarial attacks trained on diverse source environments transfer to unseen target policies. Domain randomisation across action spaces, reward functions, transition dynamics, policy algorithms, and network structures enables black-box or gray-box attacks without knowledge of target specifics. Interestingly, attacks trained on randomised domains sometimes outperform white-box attacks on the target, suggesting that robust attacks develop from exposure to environmental diversity [55].

Adversarial Minority Influence (Black-Box): AMI operates without model access, learning attack policies through repeated interaction. The unilateral influence filter enables the attacker to focus on influencing victims while disregarding reverse effects, making the attack practical in black-box settings [42].

2.4.7 FGSM

Fast Gradient Sign Method (FGSM) [13] is an efficient single-step gradient based technique for generating adversarial examples, it leverages the adaptive nature of neural networks to create small perturbations that maximise the loss increase. It is defined as follows:

$$\eta = \varepsilon \cdot \text{sign}(\nabla_x J(\theta, x, y)) \quad (2.5)$$

where:

- θ denotes the model parameters
- x represents the input
- y is the target label
- $J(\theta, x, y)$ is the cost function used to train the model

The resulting adversarial example is obtained as:

$$\tilde{x} = x + \varepsilon \cdot \text{sign}(\nabla_x J(\theta, x, y)) \quad (2.6)$$

where ε is a small scalar controlling perturbation magnitude, chosen to remain imperceptible to human observers while remaining sufficient to fool the classifier.

This means that FGSM not only misleads models but makes them confident in the mistakes that they make, successfully "convincing" them that their missclassifications are correct with a significant success rate [32]. Goodfellow et al. [13] propose that FGSM's efficacy arises from the inherent linearity of neural networks in high-dimensional spaces, fundamentally contradicting the prevailing hypothesis that adversarial vulnerabilities stem from deep network non-linearity.

FGSM efficiency and simplicity have made it a widely used baseline for development of a wide range of sophisticated adversarial attack methods [56].

Rozsa et al. (2016) [57] extends FGSM by introducing the hot/cold (HC) approach, the hot/cold method operates at the penultimate layer, constructing a feature vector that simultaneously boosts value of features of a target "hot" class and reduces value of features of the original "cold" class enabling backpropagation to generate perturbations that push samples toward different target classes.

Kurakin et al. (2016) [58] extend FGSM into the Basic Iterative Method (BIM), commonly referred to as the Iterative Fast Gradient Sign Method (I-FGSM). Rather than taking a single step of size ϵ , BIM applies FGSM repeatedly with a smaller step size α and clips the intermediate result after every step, so that the adversarial image stays within an ϵ -neighbourhood of the original:

$$\tilde{x}_0 = x, \quad \tilde{x}_{N+1} = \text{Clip}_{x,\epsilon} \left\{ \tilde{x}_N + \alpha \cdot \text{sign}(\nabla_x J(\tilde{x}_N, y_{\text{true}})) \right\} \quad (2.7)$$

The same work introduces the Iterative Least-Likely Class Method, a targeted variant that instead steps towards the class the network considers least likely for the clean input [58].

In follow-up work, Kurakin et al. (2017) [59] scale adversarial training to ImageNet. Adversarial examples are generated on the fly from clean samples using the current network state and mixed into each minibatch alongside clean examples, with a weighted loss balancing the two contributions.

Xie et al. (2019) [60] proposes Diverse Inputs Iterative Fast Gradient Sign Method (DI²-FGSM), a principled extension of FGSM that addresses the white-box-black-box transferability trade-off inherent in the FGSM family of attacks. Rather than directly maximising the loss with respect to original inputs, as in I-FGSM [58], DI²-FGSM applies stochastic, differentiable transformations (random resizing and padding) at each iteration with probability p , maximising loss with respect to these transformed inputs.

2.4.8 PGD

Projected Gradient Descent (PGD) is an iterative adversarial attack method that extends FGSM by applying multiple smaller perturbation steps. Projected Gradient Descent (PGD) is introduced by Madry et al. [14] as a multi-step, adversarial attack that iteratively updates an input within a constrained perturbation set to maximise loss. Concretely, for an allowed perturbation set S (typically an ℓ_∞ -ball around a clean input x), PGD starts from an initial point $x_0 \in x + S$ and repeatedly applies a signed gradient step on the loss with respect to

the input, followed by projection back onto $x + S$. Madry et al. describe this as projected gradient descent on the negative loss function, which is equivalent to projected gradient ascent on the loss:

$$x_{t+1} = \Pi_{x+S}(x_t + \alpha \text{sign}(\nabla_x L(\theta, x_t, y))),$$

thereby seeking an adversarial example that attains high loss while respecting the perturbation budget. The perturbation set is defined by a norm constraint on the difference between the clean and adversarial inputs. Madry et al. focus primarily on the ℓ_∞ case, with additional experiments under ℓ_2 :

- ℓ_∞ : bounds the largest change applied to any single input feature, so every pixel may move by at most ε . Under this constraint PGD is the multi-step variant of FGSM, which corresponds to a single step of size ε .
- ℓ_2 : bounds the Euclidean distance between the original and adversarial inputs, allowing larger changes to a few features provided the overall perturbation remains small.

Other norms such as ℓ_1 , which bounds the sum of absolute changes, are equally compatible with the projected-gradient formulation but are not studied in that work.

Through extensive experiments with random restarts, the authors demonstrate that PGD reliably finds local maxima whose loss values are tightly concentrated, and thus regard PGD as a "universal first-order adversary", meaning that robustness to PGD is indicative of robustness to any attack that only uses first-order (gradient) information [14].

A further pitfall of single-step adversaries is "label leaking", identified by Kurakin et al. (2017) [59] and highlighted in the survey of Akhtar et al. (2021) [56]. When a model is adversarially trained on FGSM examples it can come to perform better on adversarial examples than on clean ones. Kurakin et al. attribute this to the construction process using the true label, which allows the model to learn regularities in how the examples are built rather than to become genuinely robust; they recommend replacing the true label with the label predicted by the model [59]. Madry et al. observe the same behaviour when training with FGSM and trace it to the fact that such an adversary produces a very restricted set of adversarial examples that the network can overfit to. The resulting networks perform poorly on natural examples and show no robustness against PGD adversaries, which supports their broader argument that robustness must be evaluated against a strong multi-step attacker rather than a single-step one [14].

PGD is the inspiration for the timing mechanism of the attack developed in this work (Section 3.3.3). Where FGSM spends its entire budget in one step and PGD divides it into several smaller steps, the attack divides its effort across time and acts on only a fraction of the time steps. The analogy concerns *when* the adversary acts rather than how strongly: every step on which it does act is still bounded by the full ε .

2.4.9 Overview of adversarial attacks

This subsection is for a summary and ease of reference of the attacks mentioned previously. This is by no means an exhaustive list of adversarial attacks, but should give a good overview of the state of the art.

Table 2.1: Taxonomy of Adversarial Attacks on MADDPG

Attack Domain	Attack Name	Mechanism	Efficacy
Observation Space	FGSM/PGD	Single/iterative gradient-based state perturbations	Limited effectiveness; imperfect critic gradients
	SAJA	Two-phase gradient ascent on state then action; loss combines $-Q$ and action difference	9.90% reward drop (3a-FACMAC); stronger than state-only or action-only attacks; bypasses M3DDPG, PAAD, ATLA
	PA-AD	Policy Adversary MDP with director-actor collaboration	SOTA on Atari/MuJoCo; outperforms FGSM/PGD
Action Space	Direct Action Manipulation	Gradient ascent on critic for action perturbations	Effective with partial actuator control; synergistic with state attacks
	Wolfpack Attack	Coordinated multi-stage targeting; exploit inter-agent dependencies	Up to 51.8% reward reduction; severe SMAC impact
	AMI (Adversarial Minority Influence)	Unilateral influence filter + targeted adversarial oracle	Attacks robot swarms, StarCraft II, MuJoCo; black-box capable
Communication Space	Message Hijacking	Intercept and modify inter-agent messages	$> 4\times$ performance degradation (simulated predator-prey); $\approx 2\times$ (physical robots on Robotarium)
	Adaptive Communication Attacks	Learn persistent coordination disruption strategies	Bypasses privacy mechanisms; adaptive to learned patterns

Table 2.2: Taxonomy of Adversarial Attacks on MADDPG part 2

Attack Do-main	Attack Name	Mechanism	Efficacy
Poisoning & Backdoor	Model Poisoning (Federated)	Attackers coordinate via MADDPG to craft poisoned global updates	Outperforms baseline poisoning; coordinated attack via MDP
	Backdoor Attacks	Embed imperceptible triggers during training; execute on trigger	ASR $\approx 99\%$; minimal utility loss on benign tasks
	PNAct (Safe RL Backdoor)	Positive/negative action samples toggling safe/unsafe behaviour	Stealthy; increases safety violations while maintaining reward
Transfer & Black-Box	Domain Randomisation Transfer	Train on diverse environments; generalise to unseen targets	Transfers across policies, networks, action spaces; sometimes outperforms white-box
	AMI (Black-Box)	Learn attack policies via repeated interaction without model access	No model knowledge required; effective on real swarms

2.5 Graph Neural Networks

Graph Neural Networks (GNNs) are a family of deep learning models designed to operate directly on graph-structured data by iteratively propagating and aggregating information along edges, so that each node (or graph) learns a representation that encodes both its own attributes and the topology of its neighbourhood. Their core ideas are based on these principles: Local connectivity, weight sharing, and hierarchical feature extraction. Modern surveys typically define GNNs as neural architectures that take a graph as input and learn task-specific node, edge, or graph-level outputs via repeated “message passing” or neighbourhood aggregation, distinguishing them from shallow network embedding methods that merely learn fixed vector lookups without parameter sharing across nodes [61–64].

Across the four surveys, the most common taxonomy divides GNNs into recurrent GNNs, convolutional GNNs, graph autoencoders, and spatio-temporal GNNs (with attention-based and message-passing variants largely falling under the convolutional category). Recurrent GNNs (often called “GNNs” in the early literature) define node states via recursive equations that iteratively update each node’s hidden representation using its own previous state and the states of its neighbours until a fixed point is reached, followed by a task-specific readout function. Their main advantage is conceptual generality: they naturally handle arbitrary graph structures (directed/undirected, cyclic/acyclic) and can, in principle, approximate complex relational dependencies by iterating message

passing to convergence. However, they are computationally expensive, require contraction conditions for guaranteed convergence, and are difficult to scale to large graphs because many recurrent steps and storage of intermediate states are needed; these limitations largely motivated the shift to feedforward, layer-stacked convolutional GNNs [61–64].

Convolutional GNNs (CGNNs) extend convolution from regular grids to graphs by defining each layer as a localised aggregation over a node’s neighbourhood [65, 66]. In the spectral view, this operation is grounded in graph signal processing, with explicit notions of frequency, smoothness, and graph filters. Polynomial filter approximations lead to spatially localised convolutions and practical implementations, but exact spectral methods are typically expensive and often restricted to transductive settings on a fixed graph [61–64]. As a result, classical spectral CGNNs offer strong theoretical foundations but are less natural to transfer across graphs with different topologies.

Spatial CGNNs (such as GraphSAGE and message-passing neural networks) instead define convolutions directly in the node domain by aggregating neighbour features with permutation-invariant functions (e.g., mean, sum, max, attention), often combined with neighbourhood sampling to control complexity [67, 68]. Because the aggregation functions are shared across nodes and graphs, these models scale better to large graphs and naturally support inductive generalisation to unseen nodes or entirely new graphs [61–64]. At the same time, deep stacks of spatial layers can suffer from over-smoothing, large-graph performance can depend heavily on the sampling strategy, and the theoretical analysis is generally less mature than in strictly spectral formulations.

Graph autoencoders (GAEs) form another major family. An encoder GNN maps nodes or graphs into a latent space, and a decoder attempts to reconstruct structural information such as edges or the adjacency matrix, often under an unsupervised or self-supervised loss [69, 70]. This view unifies many earlier network embedding methods within a single deep architecture: the learned representations capture topology and attributes and can then be reused for downstream tasks such as clustering, link prediction, or node classification [61–64]. GAEs are particularly useful when labels are scarce, and variational extensions allow modeling distributions over graphs. Their main drawbacks are the cost of decoders that operate on dense adjacency matrices, the risk that pure reconstruction objectives over-prioritise structural fidelity over task-relevant semantics, and sensitivity to design choices such as negative sampling and the balance between observed and unobserved edges.

Spatio temporal GNNs (STGNNs) extend these architectures to graphs with time varying signals or topology by combining graph convolutions with temporal models such as recurrent networks, temporal convolutions, or attention mechanisms [71, 72]. They are now a standard choice for applications like traffic forecasting and human action recognition, where the graph structure is relatively stable but node features evolve over time [61, 63]. The main benefit is that STGNNs can model how information propagates through both space and time—for example, how congestion spreads across a road network—in an end-to-end trainable way. The cost is increased architectural and computational complexity, a greater

need for labelled time-series data, and more challenging regularisation and interpretability due to the combined depth in the spatial and temporal dimensions.

Overall, recent surveys converge on the view that GNNs are message-passing architectures on graphs, with recurrent, convolutional, autoencoding, and spatio-temporal variants offering different trade-offs between expressivity, scalability, transferability, and ease of training [61–64].

2.5.1 GNN applications in network routing

Graph Neural Networks (GNNs) have emerged as a promising tool for learning routing and traffic engineering (TE) policies directly from network structure and traffic, with an explicit goal of generalising across topologies and traffic conditions. Existing work can be organised into three main application axes: (i) GNNs as performance models for routing, (ii) GNN+RL controllers for online routing/TE, and (iii) specialised, generalizable GNN architectures for large-scale TE and domain-specific routing [73].

Early work focuses on using GNNs as surrogate models of network behaviour, predicting performance metrics for given topologies, traffic variations, and routing configurations. Rusek et al. (2019) [74] show that message-passing GNNs can accurately estimate per-link delay and loss, enabling fast evaluation of many candidate routing schemes without full simulation. Subsequent studies, summarised in Jiang et al. (2021) [73], extend this idea by feeding GNN predictions into external optimisers or discovery based search procedures, transforming TE into a learn, predict, optimise loop. This line of work primarily treats routing as a supervised regression problem, with GNNs learning a differentiable mapping from graph + routing configuration to performance.

A second stream couples GNN encoders with deep reinforcement learning (DRL) to directly learn routing or TE policies. G-Routing proposes a flexible online routing framework where a GNN encodes the current topology, traffic matrix, and routing state, predicting future path-level metrics that a DRL agent then uses to choose routes [75]. Experiments show that G-Routing converges faster and yields lower jitter and more stable performance than shortest-path and traditional TE baselines under traffic dynamics and failures [75]. Similarly, GROM (Generalized Routing Optimization Method) uses a GNN+DRL design to learn traffic-splitting policies; the authors encode split ratios as link features and discretise the continuous action space into classification, improving training stability and generalisation to unseen topologies [76].

More recent works [77] explicitly target generalisation across networks and demand patterns by designing GNN architectures aligned with classical TE formulations. A representative approach constructs a TE-GNN whose graph includes paths, source–destination demand pairs, links, and a global objective node, effectively embedding the linear program structure of TE into a heterogeneous graph. Message passing over this graph allows the GNN to learn correlations between paths, links, traffic and the global objective (e.g., maximum link utilisation), and experiments indicate that such models can generalise TE

policies to unseen topologies while approximately respecting constraints [77]. Jiang et al. (2024) [78] broader survey situates these models within a larger family of GNN-based control and optimisation methods, highlighting that TE and routing are a well studied application for GNNs.

2.6 Taxonomy on Adversarial attacks on GNNs

This subchapter will present contemporary research on adversarial attacks against Graph Neural Networks (GNNs), characterising them in six categories. The classification scheme addresses the unique challenges posed by graph-structured data and the message-passing mechanisms of GNNs that attackers must address in their strategies.

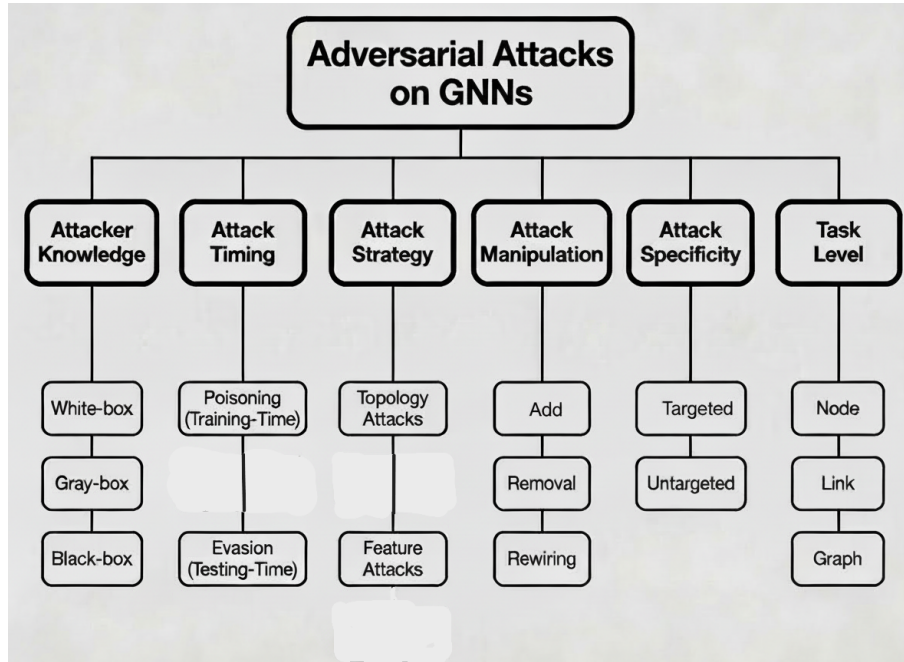


Figure 2.1: Diagram of Adversarial Attack characteristics on Graph Neural Networks

2.6.1 Attacker knowledge

Adversarial attacks on GNNs can be categorised based on the attacker's knowledge:

Table 2.3: Attacker’s Knowledge (Information Access Level)

White-Box	Gray-Box	Black-Box
Complete model access: parameters, architecture, training data, labels	Limited access: training data and labels available; model parameters obscured	Minimal access: only adjacency matrix, feature matrix, and query access to model outputs
Gradient-based optimisation feasible; represents worst-case vulnerability assessment	Requires surrogate model training	Query-efficient methods: reinforcement learning, genetic algorithms, or transfer from surrogate models

2.6.2 Attack Timing

Adversarial attacks on GNNs can be categorised by when they are deployed:

- **Training-Time Attacks (Poisoning Attacks):** Adversarial examples are injected into training data to manipulate the model’s learned representations [79, 80].
- **Test-Time Attacks (Evasion Attacks):** Adversarial examples are introduced during test time to interfere with model predictions [79, 80].

2.6.3 Attack Strategy (Perturbation Goal)

Adversarial attacks on GNNs can be categorised based on their perturbation goals:

- **Topology Attacks:** Manipulate the graph structure by adding or removing edges to mislead the GNN.
- **Feature Attacks:** Alter node features to deceive the GNN’s predictions.
- **Hybrid Attacks:** Combine topology and feature perturbations to maximise attack effectiveness.

2.6.4 Attack Manipulation

Adversarial attacks on GNNs can be categorised based on the manipulation method whose effectiveness is generally evaluated by stealth rate while maintaining success rates:

- **Addition:** Introduce new nodes or edges to the graph.
- **Removal:** Delete existing nodes or edges from the graph.
- **Rewiring:** A mix of addition and removal.

Rewiring operations tend to preserve graph statistics (node count, edge count, degree distribution) while enabling structural perturbations, making them significantly less detectable [81].

2.6.5 Attack Specificity

Adversarial attacks on GNNs can be categorised based on their specificity:

- **Targeted Attacks:** Aim to degrade specific model functions or characteristics.
- **Untargeted Attacks:** Seek to degrade overall model performance without targeting specific functions.

2.6.6 Task level

Adversarial attacks on GNNs can be categorised based on the level in which they operate:

- **Node-Level Attacks:** Target individual nodes to mislead node classification or regression tasks. A key vulnerability at this level in GNNs is the assumption of local homophily [79].
- **Link-Level Attacks:** Focus on specific edges to disrupt link prediction tasks. At this level a common problem is the triadic closure disruption where nodes sometimes create links between them despite having another node which connects them, effectively creating a "triangle" [81]
- **Graph-Level Attacks:** Aim to alter entire graph structures to affect graph classification tasks. An exploitable vulnerability at this level is the graph pooling instability [81].

2.6.7 Examples and characteristics of adversarial attacks on GNNs

This is by no means a thorough list of adversarial attacks on GNNs, but should give a good overview of the state of the art.

Table 2.4: Summary of Adversarial Attacks on GNNs

Attack Method	Knowledge	Capability	Strategy	Manipulation	Goal	Task
Nettack [82]	Gray-box	Both	Hybrid	Rewiring	Targeted	Node
Metattack [83]	Gray-box	Poisoning	Topology	Rewiring	Untargeted	Node
FGA [84]	White-box	Evasion	Topology	Rewiring	Targeted	Node
RL-S2V [85]	Black-box	Evasion	Topology	Rewiring (edge flips)	Targeted	Node/Graph
PGD-Attack [86]	White-box	Evasion	Topology	Rewiring	Both	Node
NIPA [87]	Gray-box	Poisoning	Hybrid	Inject nodes	Untargeted	Node
ReWatt [88]	Black-box	Evasion	Topology	Rewiring	Untargeted	Graph
IG-FGSM [89]	White-box	Evasion	Hybrid	Rewiring(Feature+Edge)	Both	Node

[79, 80]

Although both are gradient-based, Nettack and Metattack are gray-box rather than white-box attacks in their original formulations. Nettack first attacks a linearised surrogate model that the attacker trains on the clean graph, and so does not require the target model's parameters; it perturbs both the graph structure and the node features, and is evaluated in both evasion and poisoning settings [82]. Metattack is a training-time poisoning attack that

perturbs the graph structure using meta-gradients and likewise assumes no knowledge of, or access to, the target classifier [83]. In both cases the optimised perturbations are transferred to the victim model, which is why they are classified as gray-box [79, 80].

2.6.8 Transferability-Focused Attacks

Recent work demonstrates that adversarial examples transfer across GNN architectures, datasets, and even task levels. GOTack leverages graph orbit learning to generate universal perturbations effective against multiple GNN variants. Migratability studies show 70-85% attack success rate when transferring between GCN, GAT, and GraphSAGE [90, 91].

METHODOLOGY

Companion studies and have shown that a MADDPG based network is robust against FGSM style attacks, individual agents attacking all links in a network during every step is ineffective despite having white-box access to reward and critic structures. We investigate whether a stronger attack is more effective, through two mechanisms. The first coordinates the perturbation across the compromised agents instead of attacking each agent individually, and is inspired by SAJA [18]. The second times the attack so that it acts on only a fraction of the steps instead of every step, and is inspired by PGD [14]. The attack is realised as a learned adversary that combines both mechanisms.

3.1 Starting Point and Scope of This Work

This work is the third stage of a line of research, and it is useful to state at the outset which components are inherited and which are contributed here, since the research question is comparative and therefore depends on the earlier stages remaining unchanged.

3.1.1 The routing system under test

The victim is a multi-agent routing controller developed in earlier work within the same research line [17]. It applies Multi-Agent Deep Deterministic Policy Gradient (MADDPG) to adaptive packet routing under the paradigm of centralised training with decentralised execution (CTDE) [5]: each agent i holds an actor π_{θ_i} that conditions only on its own local observation, while its critic Q_{ϕ^i} may, during training, observe the joint state and the actions of every other agent. Conditioning the critic on the joint action is what stabilises learning, since from any single agent’s perspective the environment would otherwise appear non-stationary as the other agents’ policies evolve [5]. After training the critics are discarded and each agent acts on local information alone. In this routing formulation every forwarding node is an agent whose observation summarises local link utilisation, queue occupancy and the destinations awaiting service, and whose action selects, for each destination, one of K pre-computed candidate paths; the trained system is thus a distributed controller in which no node holds a global view, yet the paths chosen

collectively determine end-to-end delivery. That prior work also investigated critic design and graph-based state encoding, which is the origin of the variant family evaluated in Section 3.2.

This thesis does not modify the routing controller. The trained agents are treated as a fixed system under test: their parameters are frozen, and the adversary interacts with them only through forward evaluations and gradients taken with respect to their inputs.

3.1.2 The FGSM baseline

The second stage established the adversarial setting. A single-step attack based on the Fast Gradient Sign Method (FGSM) [13] was developed against these agents in a preceding dissertation in the same line of work [TODO-colleague-fgsm]. FGSM perturbs an input one step along the sign of the gradient of a loss with respect to that input, $\eta = \varepsilon \cdot \text{sign}(\nabla_x J(\theta, x, y))$, under an ℓ_∞ budget $\|\eta\|_\infty \leq \varepsilon$. Applied to the routing controller, the victim is the trained actor, the input is an agent’s local observation, and the objective rewards actions that degrade routing performance rather than ones that induce a misclassification. The threat model it fixed is *observation-space* rather than *actuation-space*: the adversary corrupts what the agents perceive, modelling a compromised telemetry or monitoring plane, while the network itself continues to evolve according to its true state.

That stage contributed not only an attack but an evaluation protocol — the perturbation budget sweep, the frozen-victim assumption, and the metrics by which an attack is judged — and it returned a negative result. The FGSM perturbation alters a substantial fraction of the routing decisions taken by the victim, yet recovers only a small part of the degradation achievable in principle, because the redundancy of the K -path topology absorbs the redirected traffic and leaves end-to-end delivery largely intact.

3.1.3 The learned-adversary scaffold

The attack of this thesis is built on an existing learned-adversary implementation [17], provided as the starting point for this project. It trains a network against the frozen victim to perturb the agents’ observations, learning from the damage its perturbations cause (Section 2.4.1), and plugs into the FGSM baseline’s evaluation loop. In that form it attacked every agent independently and at every step, the same pattern as the baseline. This work uses it as the vehicle for its two mechanisms: the SAJA-inspired coordination and the PGD-inspired timing described in Section 3.3 were built into it, together with the random control used to score it.

3.1.4 Contribution of this thesis

That negative result admits two readings: either the controller is genuinely robust, or a single-step, per-agent adversary is too weak for its failure to constitute evidence of robustness. Distinguishing between them requires a stronger adversary, and supplying one is the contribution of this work. The attack addresses the two weaknesses in turn,

drawing on a different source of inspiration for each. Where FGSM spends its whole budget in a single step, PGD divides it into smaller projected steps [14]. The attack carries that principle into time: it acts on only a fraction of the time steps, concentrating its budget on high-leverage moments rather than applying a full-strength perturbation at every step (Section 3.3.3). Where the baseline perturbs each agent in isolation, SAJA [18] judges the perturbations of several victim agents together, through the centralised critic’s value of the joint action they induce. The attack likewise coordinates the perturbation jointly across the compromised agents (Section 3.3.2). It is realised by extending the learned-adversary scaffold of Section 3.1.3, trained against the frozen victim to combine both mechanisms (Section 3.3.4).

Because the purpose is comparison rather than a stronger attack in isolation, compatibility with the earlier stage is a design constraint rather than a convenience. The attack operates on the same frozen agents, within the same ℓ_∞ ball, and is scored under the same evaluation protocol as the FGSM baseline. Any difference in measured damage is then attributable to the attack itself, rather than to a change in the adversary’s capabilities or in the conditions of measurement. The domain constraint is the one respect in which the evaluation goes beyond this: the attack is evaluated under the baseline’s constraint, which clamps only the bandwidth features, and additionally under a stricter one that clamps every observation feature. Only results under the former are directly comparable with the FGSM study (Section 3.3.4). Where the constraint forced other implementation choices, they are noted at the relevant point in the text.

3.2 Victim Architectures Under Test

The attacks are evaluated against the family of trained variants inherited from the base system [17] rather than against a single model, so that conclusions about the routing task can be separated from conclusions about one particular network design. All variants share the same actor: a multilayer perceptron mapping the local observation to n_d blocks of K path scores, with the executed route given by the 2 max within each block. They differ along three binary axes, described below.

3.2.1 Central (CC) versus local (LC) critic

The two critic domains differ in what the value function is allowed to observe during training:

$$\underbrace{Q_{\phi^i}(\mathbf{s}, a_1, \dots, a_N)}_{\text{CC: } N(d_{\text{obs}} + |\mathcal{A}|)} \quad \text{versus} \quad \underbrace{Q_{\phi^i}(s_i^{\text{local}}, a_i)}_{\text{LC: } d_{\text{obs}} + |\mathcal{A}|}, \quad (3.1)$$

with d_{obs} the per-agent observation dimension and $|\mathcal{A}| = n_d K$ the per-agent action dimension. The central critic is the canonical MADDPG construction [5]: conditioning on the joint action removes the non-stationarity each agent would otherwise face as its peers’ policies change, at the cost of an input that grows linearly in the number of agents.

The local critic reduces the scheme to independent actor–critic learners; it forfeits that guarantee but is far cheaper and its size is independent of the agent count.

The distinction also determines which attack objectives are available. Because the central critic is an explicit function of *all* agents’ actions, it supplies a non-separable objective that couples the compromised agents, used in Equation (3.8). Local-critic variants admit no such objective and are attacked with the congestion objective of Equation (3.9) instead.

3.2.2 Simple versus duelling value head

The simple head passes the concatenated state and action through two rectified hidden layers and emits a scalar. The duelling head replaces the output layer with two parallel streams whose outputs are summed, $Q(s, a) = V(h) + A(h)$.

One caveat should be stated. In the original duelling formulation the advantage stream emits one value *per action*, and subtracting the mean advantage makes the split identifiable. Here the action is an *input* to the critic rather than an output index, so both streams emit scalars and no such constraint is applied: any constant may be shifted between V and A without changing Q . The duelling head is thus an extra parallel projection rather than a value–advantage separation, and differences it produces should be attributed on that basis.

3.2.3 With and without GNN encoding

The GNN variants insert a shared encoder between the environment and the actor. Each agent is a graph node whose features are its full observation, edges come from the trainable subgraph of the physical topology, and two GraphConv layers aggregate neighbour information under a residual connection:

$$\tilde{\mathbf{X}} = \text{ReLU}\left(\text{GraphConv}_2(\text{ReLU}(\text{GraphConv}_1(\mathbf{X}, E)), E) + \mathbf{X}\right). \quad (3.2)$$

The encoder is dimension-preserving, so it is a drop-in pre-processor and the actor is unchanged between variants, and the residual connection makes it a near-identity transform at initialisation. Optional relay nodes representing transit switches carry zero features and act purely as message-passing intermediaries, propagating observations two hops through the topology.

The consequence for the attacks is structural rather than one of degree: without the encoder an agent’s policy reads its own observation alone, whereas with it a perturbation applied to one agent reaches its neighbours’ encoded states before their actors. The single-agent differentiable path assumed by the critic-grounded attack therefore does not hold for these variants, which must be attacked with the joint formulation of Section 3.3.2 or excluded from that comparison.

Table 3.1: MADDPG victim variants under test, named <critic domain>-<value head>[-GNN].

Variant	Critic	Value head	GNN
CC-Simple	Central	Simple	—
CC-Duelling	Central	Duelling	—
CC-Simple-GNN	Central	Simple	Yes
CC-Duelling-GNN	Central	Duelling	Yes
LC-Simple	Local	Simple	—
LC-Duelling	Local	Duelling	—
LC-Duelling-GNN	Local	Duelling	Yes

3.2.4 Variant matrix

LC-Simple-GNN is omitted, as the local-critic value-head comparison is already covered by LC-Simple/LC-Duelling and the GNN comparison by LC-Duelling/LC-Duelling-GNN. Where a single variant is reported, CC-Simple serves as the reference configuration, being the canonical MADDPG formulation with no additional architectural machinery.

3.3 Coordinated and Strategically-Timed Attacks on MADDPG Routing Agents

The attack builds on two attacks reviewed in Chapter 2: SAJA (Section 2.4.2), which inspires its coordination mechanism, and PGD (Section 2.4.8), which inspires its timing mechanism and whose projected iteration also underlies the reference formulation of coordination given below. PGD can be stated formally as follows. Given a clean input x , label y , model parameters θ , loss function $L(\theta, x, y)$, a perturbation budget ε , a step size α and a norm constraint set

We adapt one idea from each attack to the MADDPG routing environment, as two features that go beyond the FGSM baseline, and realise the resulting attack as a learned adversary (Section 3.3.4) that combines them.

- **(A) Coordinated multi-agent perturbation, inspired by SAJA:** replacing the independent per-agent loop with a joint adversary whose perturbations are judged together by their combined effect, as SAJA judges victim perturbations through the centralised critic’s value of the joint action, so that several compromised agents can be directed towards a single shared link.
- **(B) Strategically-timed, critical-state attack, inspired by PGD:** dividing the attack across time, as PGD divides its budget into smaller steps, by imposing an L_0 budget on how often the adversary may act and monitoring the network with a critical-state score so that the attack is triggered at the moments when the network’s redundancy is thinnest.

3.3.1 Threat model and admissible perturbation set

The attack uses a threat model based deliberately on the one used for the FGSM baseline, so that any observed difference in damage is attributable to the attack itself rather than to a change in the attacker’s capabilities. The single addition, a stricter domain constraint under which the attack is also evaluated, is identified below.

Let $C \subseteq \{1, \dots, M\}$ denote the set of compromised agents and $N = |C|$ its cardinality. The victim policies $\{\pi_{\theta_i}\}$ and critics $\{Q_{\phi^i}\}$ are *frozen*: their parameters are fixed throughout, and the adversary interacts with them only through forward evaluations. Unlike the FGSM baseline, and unlike both SAJA and PGD, the gradient-based attacks that inspire it, it never takes gradients of the victim, even with respect to its input observations. No victim parameter is ever updated, so the attack measures the robustness of a fixed trained policy rather than the outcome of an adversarial training game.

The adversary perturbs observations only; the underlying network state evolves from the true, unperturbed dynamics. For agent i at time t the admissible set is

$$\mathcal{S}(s_{i,t}^{\text{local}}, \varepsilon) = \left\{ \hat{s} \mid \|\hat{s} - s_{i,t}^{\text{local}}\|_{\infty} \leq \varepsilon, \hat{s} \in \mathcal{D} \right\}, \quad (3.3)$$

where $\mathcal{D}$ is the domain constraint that re-projects the normalised bandwidth features onto $[0, 1]$. The corresponding projection operator $\Pi_{\mathcal{S}}$ therefore composes an ℓ_{∞} clip about the clean observation with the same domain clip used by the FGSM baseline. Because the measured damage depends on the size of the admissible set, the attack is compared against a random control (Section 3.3.5) that uses exactly the same operator. Comparing an attack with a control drawn from a different set would confound the perturbation direction with the size of the set it is drawn from. Equation (3.3) is the set inherited from the FGSM baseline, and the reference formulation of Section 3.3.2 is written in terms of it. The learned adversary is evaluated under this set and, separately, under a stricter operator that clamps every observation feature; in both cases its random control shares the operator the attack uses. The two operators and their consequences are set out in Section 3.3.4.

3.3.2 (A) Coordinated multi-agent perturbation

The coordination mechanism is inspired by SAJA [18] (Section 2.4.2). SAJA perturbs the observations of several victim agents at once and scores each candidate perturbation by the centralised critic’s value of the joint action the agents then take, so the perturbations are judged by their combined effect rather than one agent at a time. This subsection explains why that matters in a routing network, and formulates the corresponding objective for the compromised agents of the threat model above.

Separability of the per-agent attack

The FGSM baseline, and any attack that iterates over agents independently, solves N decoupled problems: for each $i \in C$,

$$\eta_{i,t}^* = 2 \max_{\eta: s_{i,t}^{\text{local}} + \eta \in \mathcal{S}} \mathcal{L}_i(s_{i,t}^{\text{local}} + \eta), \quad (3.4)$$

where $\mathcal{L}_i$ depends only on agent i 's own observation and policy. The joint perturbation obtained this way is the concatenation of N independent solutions, and the implied joint objective is *separable*,

$$\mathcal{J}_{\text{sep}}(\hat{\mathbf{s}}_t) = \sum_{i \in C} \mathcal{L}_i(\hat{s}_{i,t}), \quad (3.5)$$

so no term couples two agents. This is a structural limitation rather than a matter of attack strength: a separable objective cannot express a preference for outcomes that depend on *which* agents are steered onto the *same* resource, because the value of steering agent i onto a link is independent of whether agent j was steered onto it as well. In a topology whose robustness comes from K -path redundancy, the damaging configurations are precisely those in which several redirected flows converge on one surviving link, and these lie outside the reach of Equation (3.5).

Joint formulation

We therefore optimise over the joint perturbation directly. Writing $\mathbf{s}_t = (s_{1,t}^{\text{local}}, \dots, s_{N,t}^{\text{local}})$ for the stacked observations of the compromised agents, the coordinated attack solves

$$\hat{\mathbf{s}}_t^* = 2 \max_{\hat{\mathbf{s}} \in \mathcal{S}(\mathbf{s}_t, \epsilon)^N} \mathcal{J}(\hat{\mathbf{s}}), \quad (3.6)$$

with a *non-separable* global objective $\mathcal{J}$. A single gradient of $\mathcal{J}$ supplies a descent direction for every compromised agent simultaneously, so one optimisation step trades the agents off against one another.

The remainder of this subsection sets out how Equation (3.6) would be solved directly, in the manner of SAJA's state phase: iterated sign-gradient ascent on an objective scored through the victim's critic, projected back onto the admissible set after every step as in PGD. This explicit formulation is presented as a reference for what coordination means, not as an attack evaluated in its own right. The adversary actually deployed is the learned adversary of Section 3.3.4. It acts in the same coordinated and projected manner, but it learns which perturbation to apply from the damage its perturbations cause, rather than solving this optimisation by differentiating the victim at every step.

Differentiating through the routing decision

The environment decodes a routing decision by taking, for each destination d , the 2 max over the K candidate paths. The realised route is therefore a piecewise constant function of the observation, and its gradient vanishes almost everywhere, which would leave

Equation (3.6) without usable first-order information. We adopt a straight-through estimator: the forward pass uses the hard per-destination one-hot decision actually executed by the environment, while the backward pass propagates the gradient of the underlying softmax policy,

$$\text{ST}(\pi_{\theta_i}(\hat{s})) = \text{sg} \left[\text{onehot}(\pi_{\theta_i}(\hat{s})) - \pi_{\theta_i}(\hat{s}) \right] + \pi_{\theta_i}(\hat{s}), \quad (3.7)$$

where $\text{sg}[\cdot]$ denotes the stop-gradient operator and $\text{onehot}(\cdot)$ is applied blockwise, one block of K entries per destination. The distinction this exposes matters: the gradient is informative about how to *change a routing decision*, which is not the same quantity as how to *cause congestion*, and the two coincide only when the network cannot absorb the redirected flow.

Attack objectives

Two instantiations of $\mathcal{J}$ are considered. The first is grounded in the victim’s own centralised critics, evaluated on the joint action induced by the perturbed observations,

$$\mathcal{J}_{\text{critic}}(\hat{\mathbf{s}}) = -\frac{1}{N} \sum_{i \in \mathcal{C}} \mathcal{Q}_{\phi^i} \left(x_t, (\tilde{a}_1, \dots, \tilde{a}_N) \right), \quad \tilde{a}_j = \text{ST}(\pi_{\theta_j}(\hat{s}_j)), \quad (3.8)$$

where x_t is the true centralised state, held fixed during the attack. Because the centralised critic of the CTDE formulation (Section 3.2.1) takes the actions of *all* agents as input [5], it is by construction a non-separable function of the joint perturbation, and ascending $-Q$ steers the joint action towards the region the victim’s own value model rates as worst. This is the value term of SAJA’s state loss [18], applied to all compromised agents at once and without its action-difference term.

The second objective targets the shared-bottleneck mechanism explicitly. Let $u_{d,k}^{(i)}$ denote the true bottleneck utilisation of the k -th candidate path to destination d for agent i , read from the *clean* observation, and let $[\tilde{a}_i]_{d,k}$ be the corresponding entry of the straight-through action. Then

$$\mathcal{J}_{\text{cong}}(\hat{\mathbf{s}}) = \sum_{i \in \mathcal{C}} \sum_d \sum_k [\tilde{a}_i]_{d,k} u_{d,k}^{(i)}. \quad (3.9)$$

Summing over agents rewards configurations in which several agents are steered onto the same congested path, which is the mechanism under test. The utilisation coefficients are taken from the clean observation rather than the perturbed one; were they read from $\hat{\mathbf{s}}$, the attacker would be optimising against its own forged view of the network instead of its true state.

Joint projected gradient ascent

Equation (3.6) is solved by projected gradient ascent following the standard PGD construction [14], which is also how SAJA computes its state perturbation [18], applied to the

stacked perturbation. Starting from a uniformly random point inside the ball,

$$\hat{\mathbf{s}}^{(0)} = \Pi_{\mathcal{S}}(\mathbf{s}_t + \delta_0), \quad \delta_0 \sim \text{Unif}(\mathcal{S}(0, \varepsilon)^N), \quad (3.10)$$

$$\hat{\mathbf{s}}^{(\tau+1)} = \Pi_{\mathcal{S}}\left(\hat{\mathbf{s}}^{(\tau)} + \alpha \cdot \text{sign}(\nabla_{\hat{\mathbf{s}}} \mathcal{J}(\hat{\mathbf{s}}^{(\tau)}))\right). \quad (3.11)$$

The random initialisation is what distinguishes PGD from the iterative FGSM variant, which starts from the clean point [58]; the procedure is repeated from R independent random starts and the iterate achieving the highest objective is retained, since the local maxima of the inner problem are distinct but concentrated in value [14]. The step size defaults to $\alpha = 2.5 \varepsilon / T$ for T iterations, which guarantees the boundary of the ball is reachable from any interior starting point [14]. The full procedure is given in Algorithm 1.

Algorithm 1: Coordinated multi-agent perturbation by projected gradient ascent (reference formulation, one time step)

Input: clean observations $\mathbf{s}_t$; centralised state x_t ; budget ε ; steps T ; step size α ;
 restarts R ; objective $\mathcal{J}$

Output: adversarial observations $\hat{\mathbf{s}}_{i,t}$ for all $i \in C$

$\mathcal{J}^{\text{best}} \leftarrow -\infty$

$U \leftarrow$ path utilisations read from the clean $\mathbf{s}_t$

for $r \leftarrow 1$ **to** R **do**

$\hat{\mathbf{s}} \leftarrow \Pi_{\mathcal{S}}(\mathbf{s}_t + \delta_0), \quad \delta_0 \sim \text{Unif}(\mathcal{S}(0, \varepsilon)^N) \quad // \text{ random start}$

for $\tau \leftarrow 1$ **to** T **do**

$\tilde{a}_j \leftarrow \text{ST}(\pi_{\theta_j}(\hat{\mathbf{s}}_j))$ for all $j \in C \quad // \text{ one shared graph}$

$J \leftarrow \mathcal{J}(\tilde{a}_{1:N}, x_t, U)$

$g \leftarrow \nabla_{\hat{\mathbf{s}}} J \quad // \text{ one backward} \Rightarrow \text{gradient for every agent}$

$\hat{\mathbf{s}} \leftarrow \Pi_{\mathcal{S}}(\hat{\mathbf{s}} + \alpha \text{sign}(g))$

end

if $\mathcal{J}(\hat{\mathbf{s}}) > \mathcal{J}^{\text{best}}$ **then**

$\mathcal{J}^{\text{best}} \leftarrow \mathcal{J}(\hat{\mathbf{s}}); \quad \hat{\mathbf{s}}^* \leftarrow \hat{\mathbf{s}}$

end

end

return rows of $\hat{\mathbf{s}}^*$

In this reference formulation the attack is solved once per time step for the whole compromised group, and each agent then receives its own block of the joint solution, so the N agents act on a single coordinated decision rather than on N independent ones. The learned adversary preserves this property, emitting one joint perturbation for the group, but produces it with a single forward pass of a trained policy in place of the $R \times T$ gradient evaluations of Algorithm 1.

3.3.3 (B) Strategically-timed attacks under an L_0 budget

The attacks considered so far act at every time step. A more realistic adversary is constrained in *how often* it can act, which converts the attack into a resource-allocation

problem over the trajectory.

Dividing the attack in this way is how the design carries PGD’s central idea from the input into time. FGSM spends its entire budget in a single step of size ε , and the FGSM baseline did so at every time step for every agent. PGD instead divides its budget into a sequence of smaller steps of size α . By analogy, rather than applying a full-strength perturbation at every step, the attack divides its effort across the trajectory and acts on only a fraction of the steps. The analogy concerns *when* the adversary acts, not how strongly: every step on which it does act is still bounded by the same ε . We impose an L_0 budget: for an episode of T_{ep} steps and a budget fraction $b \in (0, 1]$,

$$\frac{1}{T_{\text{ep}}} \sum_{t=1}^{T_{\text{ep}}} \mathbb{1}[\text{attack at } t] \leq b. \quad (3.12)$$

Spending this budget uniformly at random would waste it on steps at which the network has ample spare capacity. We instead spend it at moments of high leverage, identified by a *critical-state score*. We use the maximum link utilisation across the topology,

$$c_t = \max_{(u,v) \in E} \rho_t(u, v), \quad (3.13)$$

where $\rho_t(u, v)$ is the utilisation of link (u, v) at time t . This single quantity responds to both regimes of interest: congestion onset raises it directly, and the loss of a link raises it indirectly by concentrating traffic onto the surviving paths, so the score is elevated exactly when redundancy is thinnest.

Because the distribution of c_t is not known in advance and drifts within an episode, the trigger threshold is calibrated online. Let W_t be a sliding window of the most recent scores and

$$\theta_t = Q_{1-b}(\{c_{t'}\}_{t' \in W_t}), \quad (3.14)$$

where Q_{1-b} denotes the empirical $(1 - b)$ -quantile, computed from the window *before* c_t is inserted so that a step never influences its own decision.

A threshold test of the form $c_t \geq \theta_t$ is not sufficient here, because the score distribution has an *atom*: a single value taken by a large share of steps. Link utilisation is accumulated from fixed-size flow reservations, so the score returns to exactly the same values again and again. In the evaluations of Chapter 4, the atom that matters at the tightest budgets lies at $c_t = 0.90$ (Section 4.2.3). Whenever b is smaller than the fraction of steps scoring at or above an atom, the quantile falls on it, θ_t takes the atom’s value, and every tied step passes the test. The realised attack rate is then that fraction rather than b : the constraint of Equation (3.12) is violated, and any two budgets below that fraction yield the same attack, so the budget ceases to be the independent variable the experiment varies.

We therefore *ration* the ties. Writing N_t for the number of gate decisions up to and including step t , and F_{t-1} for the number of attacks already spent, the adversary acts at

step t if

$$fire_t = \begin{cases} 1, & c_t > \theta_t, \\ 1, & c_t = \theta_t \text{ and } F_{t-1} + 1 \leq b N_t, \\ 0, & \text{otherwise,} \end{cases} \quad (3.15)$$

so that steps strictly above the threshold always fire, while tied steps are admitted only for as long as the cumulative rate stays within budget. The tie-break is deterministic rather than randomised: a random rule would introduce variance between the attacked and the control arm, which are intended to differ only in the *direction* of the perturbation, and would require a separate random stream so as not to disturb the seeded traffic sequence.

The first H steps only observe. They seed the window without attacking, since the quantile is not meaningful until the window is filled and acting beforehand would spend budget the gate cannot yet justify. Those decisions still count towards N_t , so the allowance left unspent during the warm-up is recovered afterwards and the realised rate over the episode approaches b from below. Since it is the realised rate, not the requested one, that a comparison across budgets actually varies, it is measured and reported alongside every result rather than assumed equal to b . The procedure is summarised in Algorithm 2.

Algorithm 2: Critical-state timing gate with rationed ties

Input: budget b ; window $\mathcal{H}$ of size W ; warm-up length H ; running counts N (decisions) and F (attacks spent); network state at step t

Output: decision $fire \in \{\text{true}, \text{false}\}$

```

 $c_t \leftarrow \max_{(u,v) \in E} \rho_t(u,v)$  // critical-state score
 $N \leftarrow N + 1$ 
if  $|\mathcal{H}| < H$  then
    |  $fire \leftarrow \text{false}$  // warm-up: observe, do not act
else
    |  $\theta_t \leftarrow Q_{1-b}(\mathcal{H})$  // computed before inserting  $c_t$ 
    | if  $c_t > \theta_t$  then
    | |  $fire \leftarrow \text{true}$ 
    | else if  $c_t = \theta_t$  then
    | |  $fire \leftarrow (F + 1 \leq b N)$  // ration the tie
    | else
    | |  $fire \leftarrow \text{false}$ 
    | end
end
insert  $c_t$  into  $\mathcal{H}$  (ring buffer of size  $W$ )
if  $fire$  then  $F \leftarrow F + 1$ 
return  $fire$ 
    
```

The gate composes with either attack scope: it decides *whether* to act at a given step, while Section 3.3.2 decides *how* the perturbation is distributed across agents once the

decision to act has been taken. On skipped steps the agents receive their unmodified observations.

3.3.4 Learned observation-space adversary

Sections 3.3.2 and 3.3.3 describe what the attack should do. It should perturb the compromised agents jointly, keep each perturbation within the admissible set, and divide its effort across time by acting on only a fraction of the steps. The attack is realised as a learned adversary that combines both mechanisms, built on the scaffold of Section 3.1.3. Like the reference formulation, it applies a bounded perturbation that is projected onto the admissible set. In the coordinated scope it emits one joint perturbation for the whole compromised group, the SAJA-inspired mechanism, and in every scope it is gated by the critical-state rule of Algorithm 2, the PGD-inspired one. Unlike the reference formulation of Algorithm 1, it does not find that perturbation by differentiating the victim at every step. Instead it learns which perturbation to apply from the damage its perturbations cause.

That such an adversary can be learned follows from the State-Adversarial MDP of Section 2.4.1. Because the victim is frozen, its policies are fixed, which is exactly the condition of Lemma 1: against a fixed policy, finding the optimal observation adversary is itself an ordinary MDP, and can therefore be approached with standard reinforcement learning [16]. We train such an adversary with DDPG, treating the frozen victim as part of its environment. The adversary is denoted v_ω , deliberately reusing the symbol of the SA-MDP adversary, with parameters ω .

Formulation

At time t the adversary observes the clean local observation $s_{i,t}$ of a compromised agent $i \in \mathcal{C}$ and emits a perturbation direction $a_{i,t}^v = v_\omega(s_{i,t}) \in [-1, 1]^{\dim(s)}$, produced by a tanh output layer. The observation presented to the victim is

$$\hat{s}_{i,t} = \Pi_{\mathcal{S}}(s_{i,t} + \varepsilon a_{i,t}^v), \quad (3.16)$$

so that every perturbation lies in an ℓ_∞ ball of radius ε by construction. Lemma 1 takes the perturbed state itself as the adversary’s action; parameterising it instead as a bounded direction scaled by ε is an equivalent reparameterisation that keeps the actor’s output within the admissible ball without a penalty term. The victim acts on $\hat{s}_{i,t}$, while the network continues to evolve from the true state, as the SA-MDP requires.

Lemma 1 prescribes the negative of the victim’s reward as the adversary’s reward. We depart from this deliberately. The victim is trained on a shaped per-agent reward combining several utilisation-dependent terms, which is a training signal rather than the quantity whose degradation this work measures. The adversary is instead rewarded with the fraction of forwarding events at step t that end in a drop,

$$r_t = \frac{n_t^{\text{drop}}}{\max(1, n_t^{\text{fwd}})}, \quad (3.17)$$

where n_t^{fwd} counts packets forwarded by any agent at step t and n_t^{drop} those dropped. This aligns the adversary’s objective with packet delivery, the outcome evaluated in Section 3.3.5. It is a proxy rather than an exact match: because each packet contributes one forwarding event per hop, the denominator grows with path length, so the signal is diluted relative to the end-to-end packet delivery ratio on which the adversary is ultimately scored.

Training

The actor ν_ω and a critic Q_ψ^v are multilayer perceptrons with two hidden layers of 256 ReLU units. The critic scores the perturbation that was *actually applied*. Because the projection in Equation (3.16) can truncate the actor’s output, the stored direction is the effective one, $\tilde{a}_{i,t}^v = (\hat{s}_{i,t} - s_{i,t})/\varepsilon$, rather than the raw output. Transitions $(s_{i,t}, \tilde{a}_{i,t}^v, r_t, s_{i,t+1})$ are stored in a replay buffer $\mathcal{R}$, and the critic is fitted to a one-step temporal-difference target computed with slowly updated target networks $\nu_{\omega'}$ and $Q_{\psi'}^v$,

$$\mathcal{L}(\psi) = \mathbb{E}_{\mathcal{R}} \left[\left(Q_\psi^v(s, \tilde{a}^v) - r - \gamma Q_{\psi'}^v(s', \nu_{\omega'}(s')) \right)^2 \right], \quad (3.18)$$

while the actor ascends the critic’s estimate of the damage its perturbation causes,

$$\max_{\omega} \mathbb{E}_{\mathcal{R}} \left[Q_\psi^v(s, \nu_\omega(s)) \right]. \quad (3.19)$$

The episode is treated as a continuing task, so the target in Equation (3.18) is never truncated at the episode boundary. During training Gaussian noise $\xi \sim \mathcal{N}(0, \sigma^2 I)$ with $\sigma = 0.10$ is added to the actor’s output and clipped back to $[-1, 1]$. The remaining settings are $\gamma = 0.95$, learning rates of 10^{-4} for the actor and 10^{-3} for the critic, a minibatch of $m = 256$, a buffer capacity of 2×10^5 transitions, and a soft target update with rate $\lambda = 5 \times 10^{-3}$. One gradient update is performed per environment step once $\mathcal{R}$ holds at least $n_{\text{warm}} = 2000$ transitions. Each adversary is trained for 300 episodes of 256 steps at an offered load factor of 2.0 with $\varepsilon = 0.30$. The procedure is summarised in Algorithm 3.

Algorithm 3: Training the learned observation-space adversary (independent scope)**Input:** frozen victim $\{\pi_{\theta_i}\}$; radius ε ; timing budget b ; episodes E ; steps per episode T_{ep} **Output:** adversary actor v_ω initialise v_ω, Q_ψ^v ; set $\omega' \leftarrow \omega, \psi' \leftarrow \psi$; empty replay buffer $\mathcal{R}$ **for** $episode = 1, \dots, E$ **do**reset the network and observe $\{s_{i,1}\}_{i \in C}$ **for** $t = 1, \dots, T_{ep}$ **do****if** the timing gate fires at t (Algorithm 2) **then****foreach** $i \in C$ **do** $a_{i,t}^v \leftarrow \text{clip}(v_\omega(s_{i,t}) + \xi, -1, 1), \quad \xi \sim \mathcal{N}(0, \sigma^2 I)$ $\hat{s}_{i,t} \leftarrow \Pi_S(s_{i,t} + \varepsilon a_{i,t}^v); \quad \tilde{a}_{i,t}^v \leftarrow (\hat{s}_{i,t} - s_{i,t})/\varepsilon$ **end****else** $\hat{s}_{i,t} \leftarrow s_{i,t}$ for every $i \in C$ // no attack, no transition**end**the victim acts on $\{\hat{s}_{i,t}\}$; the network steps from the true state $r_t \leftarrow n_t^{\text{drop}} / \max(1, n_t^{\text{fwd}})$ **if** the gate fired **then** store $(s_{i,t}, \tilde{a}_{i,t}^v, r_t, s_{i,t+1})$ in $\mathcal{R}$ for each $i \in C$ **if** $|\mathcal{R}| \geq \max(m, n_{\text{warm}})$ **then**sample a minibatch of size m from $\mathcal{R}$ update ψ by Equation (3.18) and ω by Equation (3.19) $\omega' \leftarrow \lambda \omega + (1 - \lambda) \omega'; \quad \psi' \leftarrow \lambda \psi + (1 - \lambda) \psi'$ **end****end****end****Scope and timing**

Both extensions of this section apply to the learned adversary. In the *independent* scope a single actor, shared by all compromised agents, maps each agent's own observation to that agent's perturbation. Every agent contributes its own transition, and all transitions share the network-wide reward r_t . Each perturbation is therefore still chosen from one agent's view alone, as in the separable problem of Equation (3.4), but the shared reward trains the actor towards perturbations that degrade the network as a whole. In the *coordinated* scope of Section 3.3.2, the observations of all compromised agents are concatenated into one joint input. A single actor then emits one joint perturbation, and the critic is additionally conditioned on the global link state, namely the available bandwidth on every edge together with each agent's modal destination. This yields one transition per step for the

whole group.

The timing gate of Section 3.3.3 is applied during training as well as evaluation. On a step it skips, the victim receives clean observations and no transition is stored, but the gradient update of Algorithm 3 still runs. A tighter budget therefore reduces the number of fresh transitions without reducing the number of updates, and each stored transition is replayed correspondingly more often. The effect is compounded in the coordinated scope, which stores one transition per step rather than one per agent. Each adversary is trained once, at $b = 0.25$, the loosest budget in the sweep, so that the tighter evaluation budgets act on a subset of the states encountered during training. At evaluation the actor is used deterministically, without exploration noise, through the same interface as the FGSM baseline, so it runs in the evaluation loop of Section 3.3.5 unchanged.

Relation to the inherited threat model

The learned adversary differs from the FGSM baseline, and from the gradient-based reference formulation of Section 3.3.2, in one respect that favours realism, and it is evaluated under two admissible sets rather than the single set of Section 3.3.1. Both should be borne in mind when its results are read.

It requires strictly *less* knowledge of the victim. FGSM, SAJA and the reference formulation all differentiate the victim with respect to its input observations, whereas the learned adversary never computes a gradient of the victim. It needs only to query the victim’s actions and to interact with a simulator of the network during training. The attack is therefore available to an adversary without access to the victim’s parameters, provided it can train against a faithful model of the system.

Its admissible set depends on the domain clamp applied by the projection Π_S , and two clamps are used. The *FGSM-parity* clamp re-projects only the normalised bandwidth features onto $[0, 1]$, which is exactly the domain constraint $\mathcal{D}$ of Equation (3.3) inherited from the FGSM baseline. The *full* clamp re-projects *every* observation feature onto $[0, 1]$, which yields a strict subset of that set. Measured on sampled on-policy observations, the full clamp removes roughly a third of the available perturbation range and binds on around four fifths of feature–step pairs. Nearly all of this loss comes from the lower limit of $[0, 1]$: over half of the observation vector sits at exactly zero, where clamping removes the downward half of the ε ball.

A separate adversary is trained under each clamp and always evaluated under the clamp it was trained with. The random control is built with the same projection as the adversary it controls, so the two arms draw from one common set and the adversarial gap remains internally valid under either clamp. Only the results under the FGSM-parity clamp are directly comparable with those of the FGSM study. The smaller set of the full clamp does not make its results a lower bound on those under the parity clamp: each clamp has its own trained adversary, and Section 4.4 shows the two adversaries harming different victims.

3.3.5 Evaluation protocol

Paired arms

Each configuration is evaluated with three arms: a *clean* arm with no perturbation, a *random* control that draws a uniformly random direction from the same admissible set as the attack, and the *attack* under test. All three are run on identical episodes: the traffic sequence and any injected link failures are generated from a common seed, so episode e of one arm and episode e of another differ only in the perturbation applied. The random control draws from a dedicated random stream so that it consumes no entropy from the traffic generator, which would otherwise desynchronise the arms and invalidate the pairing.

Adversarial-specific damage

Under link failures the raw degradation relative to the clean arm is dominated by the failures themselves rather than by the attack. The primary reported quantity is therefore the *adversarial gap*, the paired per-episode difference between the random control and the attack,

$$\Delta^{\text{adv}} = \frac{1}{n} \sum_{e=1}^n \left(\text{PDR}_e^{\text{rand}} - \text{PDR}_e^{\text{atk}} \right), \quad (3.20)$$

reported with a paired 95% confidence interval, $\Delta^{\text{adv}} \pm t_{0.975, n-1} s_d / \sqrt{n}$, where s_d is the standard deviation of the per-episode differences. Differencing within episode pairs removes the episode-to-episode variance that otherwise dominates the comparison. The raw drop with respect to the clean arm is reported alongside it, but is not used as the measure of attack strength.

Decisions versus outcomes

Two families of metric are recorded and reported separately. *Decision* metrics count how much routing behaviour the attack changed, principally the action flip rate, the fraction of per-agent, per-destination 2 max path choices that differ between the clean and adversarial observations at the same time step,

$$\text{flip} = \frac{1}{|C| n_d T_{\text{ep}}} \sum_t \sum_{i \in C} \sum_d \mathbb{1} \left[2 \max_k \pi_{\theta_i}(\hat{s}_{i,t})_{d,k} \neq 2 \max_k \pi_{\theta_i}(s_{i,t}^{\text{local}})_{d,k} \right]. \quad (3.21)$$

Outcome metrics measure delivered traffic, principally end-to-end PDR. The distinction is essential in a topology with K -path redundancy, where a high flip rate accompanied by an unchanged PDR indicates that the network absorbed the redirected flows rather than that the attack succeeded. Reporting a flip rate as though it were a measure of damage would conflate the two.

STATISTICS AND RESULTS

This chapter reports what the learned observation-space adversary of Section 3.3.4 actually achieves against the victim family of Section 3.2. The question inherited from the earlier stages of this research line is narrow and comparative. The FGSM study found that perturbing every agent at every step altered a substantial fraction of routing decisions while extracting almost none of the available damage. The present work asks whether that robustness survives a stronger attacker with two mechanisms. Inspired by PGD, it divides its effort across time: rather than spending a full-strength perturbation at every step, it acts on only a fraction of the steps, choosing *when* to spend a limited budget. Inspired by SAJA, it coordinates the perturbation across agents, choosing the compromised agents' perturbations together rather than one agent at a time.

Both mechanisms are evaluated. The independent scope perturbs each agent separately, so it isolates the first mechanism, dividing the attack across time. The coordinated scope adds coordination, and comparing the two scopes on identical episodes isolates what coordination contributes. Results are reported under two domain clamps, which decide which observation features are re-projected onto $[0, 1]$ after the perturbation. Section 4.2 reports the independent scope under the *FGSM-parity* clamp, which reproduces the admissible set of the FGSM study, so that the attack is measured on the same terms as the baseline. Section 4.3 reports both scopes under the stricter *full* clamp, and Section 4.4 compares the two clamps on paired episodes.

The results are organised around three measurements. The first is how the attack's effect varies with the timing budget, that is, with the fraction of timesteps the adversary is permitted to act on; this tests the central claim of Section 3.3.3, that concentrating a fixed budget on high-utilisation moments buys more damage than spreading it uniformly. The second is the relationship between decisions changed and packets lost, which tests whether the flip rate reported by the earlier study is a meaningful proxy for damage at all. The third is the effect of coordination, measured as the paired difference between the coordinated and independent attacks.

4.1 Reported Statistic and Experimental Protocol

Every figure and table in this chapter reports the *adversarial gap* of Equation (3.20): the paired per-episode difference in end-to-end packet delivery ratio (PDR) between the random control arm and the attack arm, expressed in percentage points. The raw drop from the clean baseline is deliberately not used as the headline number. A perturbation of magnitude ε applied to an observation will disturb the controller whether or not its direction was chosen adversarially, so the clean-to-attack difference conflates the damage attributable to the attacker’s optimisation with the damage any same-sized noise would have caused. The adversarial gap isolates the former, and it is the only quantity that speaks to whether learning the perturbation direction was worth anything.

Three arms are run for each configuration: a clean arm with no perturbation, a random control drawing uniformly from the same ε -ball under the same admissible set and the same timing gate, and the attack arm. The three arms are *paired*: each of the 30 evaluation episodes uses an identical traffic sequence and identical link-failure seed across all arms, so the per-episode differences are matched and the reported 95% confidence intervals are computed on those paired differences. Without pairing, the between-episode variance in offered traffic would dominate the effect being measured and the intervals would be meaningless.

The victim is frozen throughout, as established in Section 3.3.1: neither actor nor critic parameters receive gradient updates during any attack or evaluation, so every number below describes an attack on a fixed controller rather than a training-time interaction. Evaluation uses $\varepsilon = 0.30$ under the L_∞ norm, 30 paired episodes of 256 timesteps at an offered load factor of 2.0, with no injected link failures and a fixed traffic seed. Three sets of adversaries, one per victim variant in each set, were trained as described in Section 3.3.4: an independent and a coordinated set under the full domain clamp, and an independent set under the FGSM-parity clamp. An adversary is always evaluated under the clamp it was trained with. Each was trained for 300 episodes with the timing gate set to $b = 0.25$, the loosest budget in the sweep, so that the tighter evaluation budgets act on a subset of the state distribution seen during training.

A result is treated as evidence of a real effect only when its 95% confidence interval excludes zero; such cells are marked in the tables and drawn with filled markers in the figures. A confidence interval spanning zero is reported as a null, not as proof of robustness, since each run has a finite resolution floor that depends on the variant’s paired variance. For the tightest variants this floor is approximately 0.1 pp, meaning a true effect smaller than that would be invisible at $n = 30$ and should not be claimed either way.

4.2 Results Under the FGSM-Parity Domain Clamp

Every result in this section was obtained under the *FGSM-parity domain clamp*: after the ℓ_∞ projection, only the normalised bandwidth features of each agent’s observation are

re-projected onto $[0, 1]$, and every other feature may move anywhere within the ε -ball. This is the domain constraint $\mathcal{D}$ of Equation (3.3), the one used by the FGSM study of Section 3.1.2, so the attack and its random control draw from exactly the admissible set within which the baseline perturbed. A separate set of seven independent-scope adversaries was trained for this clamp, which is recorded in each of the 35 evaluation records. The clean arm depends on neither the adversary nor the clamp, and its per-episode delivery series is bit-identical to that of the full-clamp evaluation in all 35 configurations. The coordinated scope has not been evaluated under this clamp.

This clamp is the looser of the two: every perturbation admissible under the full clamp of Section 4.3 is also admissible here. That does not make the damage measured here an upper bound on the damage measured there. Each clamp has its own trained adversary, and a learned adversary is not guaranteed to exploit a larger set at least as well as a smaller one; Section 4.4 shows that on several variants it does not.

4.2.1 Sensitivity to the timing budget

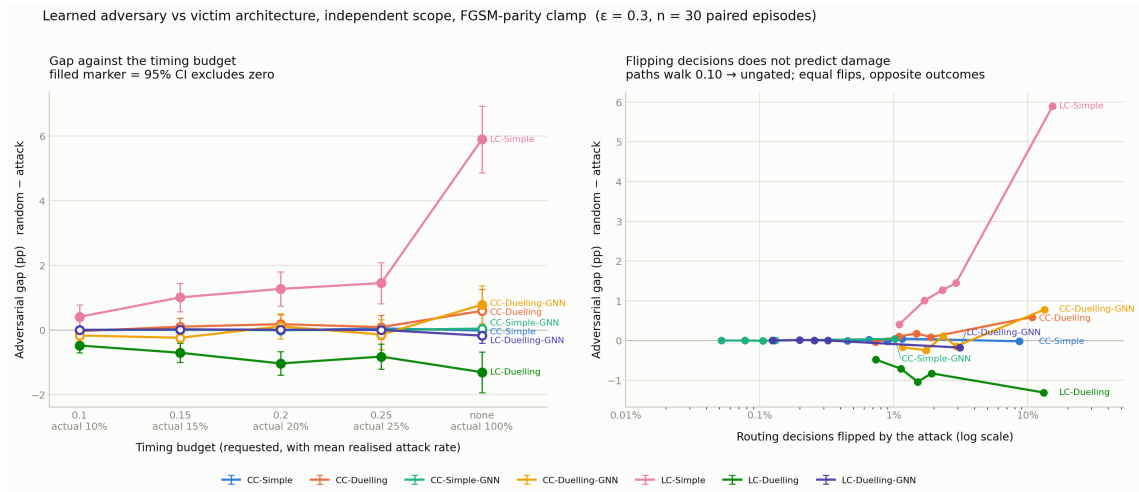


Figure 4.1: Independent scope, FGSM-parity domain clamp. Adversarial gap for all seven victim variants at $\varepsilon = 0.30$, $n = 30$ paired episodes. **Left:** gap against the requested timing budget, with the mean realised attack rate printed beneath each tick and the ungated run included as the 100% reference; filled markers denote a 95% confidence interval excluding zero, hollow markers a null. **Right:** the same gaps plotted against the fraction of routing decisions the attack actually changed, on a logarithmic axis, with each variant drawn as one path walking from $b = 0.10$ through to the ungated run. A thin line joins a label to its series where the label had to be moved clear of the data.

The timing gate spends the budget it is given: the four requested budgets realise, averaged over the seven variants, as 10.0%, 15.0%, 19.9% and 24.9% of steps. Figure 4.1 and Table 4.1 show a different architectural pattern from the one found under the full clamp. A single variant is clearly vulnerable. LC-Simple is attacked significantly at every budget, its gap rising from +0.40 pp at $b = 0.10$ to +1.45 pp at $b = 0.25$ and +5.90 pp ungated, the

largest gap measured in this study. Ungated, its delivery ratio falls to 71.8%, against 77.3% clean and 77.7% under the random control. CC-Duelling-GNN separates from zero only ungated, at +0.78 pp, and CC-Duelling does not separate at all, reaching +0.59 pp ungated with an interval that spans zero. CC-Simple, CC-Simple-GNN and LC-Duelling-GNN stay within 0.2 pp of zero at every budget.

Table 4.1: Independent scope, FGSM-parity domain clamp. Adversarial gap in percentage points by victim variant and requested timing budget, at $\varepsilon = 0.30$ and $n = 30$ paired episodes. Bold entries have a 95% confidence interval excluding zero. Positive values indicate the attack cost the victim more than the random control of the same magnitude; negative values indicate the reverse.

Variant	0.10	0.15	0.20	0.25	ungated
CC-Simple	-0.01	+0.02	-0.01	+0.05	-0.02
CC-Duelling	-0.04	+0.10	+0.18	+0.09	+0.59
CC-Simple-GNN	+0.00	+0.00	-0.01	+0.00	+0.04
CC-Duelling-GNN	-0.17	-0.24	+0.11	-0.15	+0.78
LC-Simple	+0.40	+1.01	+1.27	+1.45	+5.90
LC-Duelling	-0.48	-0.71	-1.04	-0.83	-1.31
LC-Duelling-GNN	+0.00	+0.01	+0.01	+0.00	-0.18

LC-Duelling is an exception in the other direction. Its gap is significantly *negative* at every budget, from -0.48 pp at $b = 0.10$ to -1.31 pp ungated. Ungated, the victim delivers 78.6% of packets under attack, against 77.3% under the random control and 76.9% with no perturbation at all, so the adversary trained against this victim steers it to deliver more than it would unperturbed. This is not evidence of robustness. It shows that the direction the adversary learned is systematically wrong for this victim. The adversary maximises the per-step drop fraction of Equation (3.17) rather than the end-to-end delivery ratio on which it is scored, and either the two objectives come apart on this victim or training converged to a poor direction; these runs do not separate the two.

Under this clamp, concentrating the budget on critical states does not buy more damage per attacked step. Normalising LC-Simple’s gap by the realised attack rate gives 4.0 pp per unit rate at $b = 0.10$, 6.7 at $b = 0.15$, 6.4 at $b = 0.20$ and 5.9 at $b = 0.25$, against 5.9 ungated. Beyond the tightest budget its damage grows in proportion to the number of steps attacked, and the tightest budget is the least efficient rather than the most. No other variant has a significant positive gap at any gated budget, so none shows an efficiency gain either. The efficiency found under the full clamp in Section 4.3.1 is therefore a property of the adversaries trained there, not of dividing the attack across time as such. As under the full clamp, the adversaries were trained with the gate at $b = 0.25$, so the ungated column asks them to act on states they did not encounter during training. For LC-Simple this costs nothing: its ungated gap is four times its gap at $b = 0.25$, for four times the number of steps attacked.

4.2.2 Decision flips versus delivered damage

The right panel of Figure 4.1 and Table 4.2 show the flip rate failing as a measure of damage more starkly than under the full clamp. Ungated, LC-Simple, CC-Duelling-GNN and LC-Duelling change 15.3%, 13.3% and 13.2% of routing decisions, within about two percentage points of one another, yet their gaps are +5.90, +0.78 and -1.31 pp, a spread of more than seven percentage points. The same holds at a gated budget: at $b = 0.25$, LC-Simple and CC-Duelling-GNN both change 2.92% of decisions, with gaps of +1.45 and -0.15 pp. The random control again changes many decisions on its own, between 9.6% and 10.7% on the four variants with the highest flip rates.

Table 4.2: Independent scope, FGSM-parity domain clamp. Ungated results at $\varepsilon = 0.30$, $n = 30$ paired episodes: adversarial gap with its 95% confidence interval, and the fraction of routing decisions changed by the attack and by the random control.

Variant	Gap (pp)	95% CI	Attack flips	Random flips
CC-Simple	-0.02	$[-0.17, +0.14]$	8.68%	5.23%
CC-Duelling	+0.59	$[-0.08, +1.26]$	10.77%	9.62%
CC-Simple-GNN	+0.04	$[-0.02, +0.10]$	1.02%	0.27%
CC-Duelling-GNN	+0.78	$[+0.20, +1.36]$	13.34%	10.63%
LC-Simple	+5.90	$[+4.87, +6.93]$	15.32%	10.74%
LC-Duelling	-1.31	$[-1.94, -0.68]$	13.15%	9.87%
LC-Duelling-GNN	-0.18	$[-0.42, +0.06]$	3.11%	1.11%

The two GNN variants that resist the attack show the same two behaviours as under the full clamp (Section 4.3.2). CC-Simple-GNN changes only 1.02% of decisions ungated, and at most 0.13% at any gated budget, so the perturbation rarely moves the selected path at all. LC-Duelling-GNN changes 3.11% ungated and still returns a gap of -0.18 pp: its routing does change, and the network absorbs the change.

4.2.3 What triggered the attacks

These are the first runs in which the timing gate recorded why it acted. For every attacked step it logs which case of Equation (3.15) admitted the step, a score strictly above the threshold or a tie at the threshold admitted by rationing, together with the congestion band the score fell in: saturated ($c_t \geq 1.00$), near saturation ($0.90 \leq c_t < 1.00$), high congestion ($0.75 \leq c_t < 0.90$) or moderate ($c_t < 0.75$). Figure 4.2 shows each trigger’s share of the attack arm’s attacked steps. The random control passes through the same gate, and its composition is nearly identical: at $b = 0.15$, for instance, its tie share differs from the attack arm’s by at most three percentage points for every variant.

Three findings stand out. First, no attacked step in any run, in either arm, scored 1.00 or more, or less than 0.75. The point mass in the score distribution that the rationing rule of Section 3.3.3 exists to handle lies at 0.90 rather than at full saturation. At $b = 0.10$ and $b = 0.15$ the mean threshold at which the gate fired is 0.90 for every variant, and the tied steps are the steps sitting on that point mass. Its stored value falls a floating-point

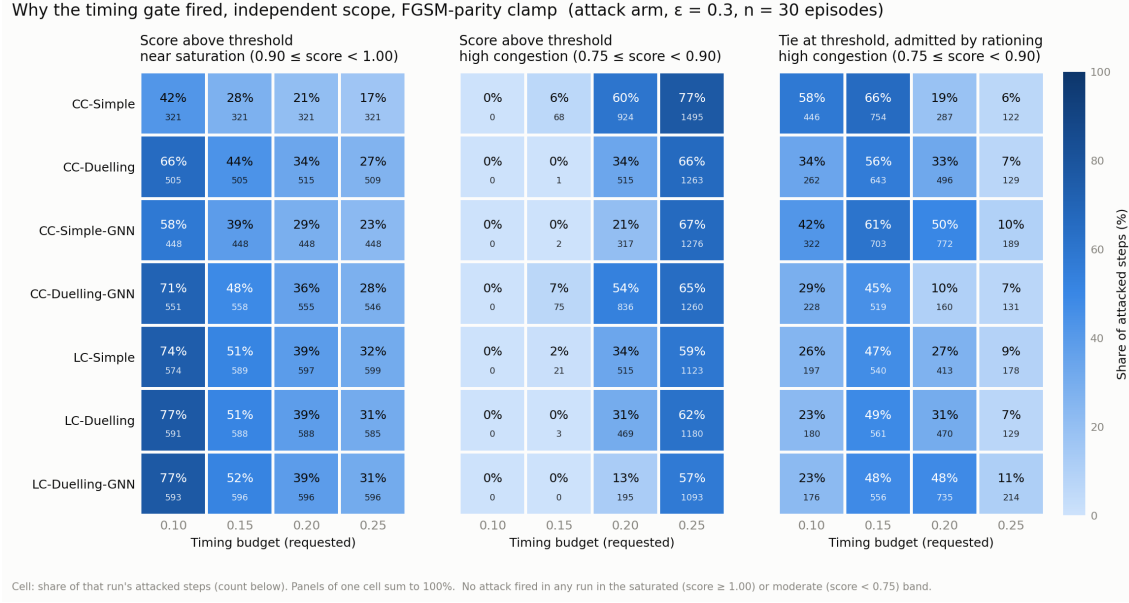


Figure 4.2: Independent scope, FGSM-parity domain clamp: why the timing gate fired in the attack arm ($\varepsilon = 0.30$, $n = 30$ episodes). One panel per trigger that occurred in at least one run, combining the case of Equation (3.15) that admitted the step with the congestion band of its score. Rows are victim variants and columns the requested timing budget. Each cell gives the trigger's share of that run's attacked steps, with the count beneath it, so the three panels of one cell sum to 100%. Ungated runs have no gate and are omitted.

rounding error below 0.90, which is why ties are recorded in the high-congestion band although they sit at that band's upper edge.

Second, the most congested states always fire. Steps scoring between 0.90 and 1.00 are admitted at every budget, in a nearly constant number per variant: 321 for CC-Simple and between 505 and 599 for the others. Their share of the attacks therefore falls from 42–77% at $b = 0.10$ to 17–32% at $b = 0.25$.

Third, the route by which the rest of the budget is spent changes with the budget. At $b = 0.10$ and $b = 0.15$ the threshold sits on the point mass, and ties admitted by rationing account for 23–58% and 45–66% of the attacks respectively. On the observed score sequences, admitting every tie would have raised the attack rate to between 17% and 21% at both budgets, collapsing the two tightest budgets into nearly the same experiment; rationing is what keeps them apart. From $b = 0.20$ the threshold begins to fall below the point mass, and by $b = 0.25$ it lies between 0.82 and 0.90. The steps on the point mass then pass as strictly above the threshold, and above-threshold steps in the high-congestion band, the point mass among them, make up 57–77% of the attacks at $b = 0.25$, while ties fall to 6–11%.

4.3 Results Under the Full Domain Clamp

Every result in this section, for both scopes and for both the attack and the random control, was obtained under the *full domain clamp*: after the ℓ_∞ projection, every feature of the perturbed observation is re-projected onto $[0, 1]$. The clamp is recorded in each evaluation record, and it is the same for all 70 configurations reported here. As set out in Section 3.3.4, this is stricter than the domain constraint of the FGSM study of Section 3.1.2, which clamped only the normalised bandwidth features. The admissible set used here is therefore a strict subset of the one used by the baseline, with roughly a third of the available perturbation range removed.

Three consequences follow. First, because the attack and its random control share the same set, the adversarial gap remains internally valid, and every comparison *within* this section is sound, including the comparison between scopes. Second, the damage measured here is not a lower bound on the damage attained under the baseline’s admissible set, even though that set is larger: the adversaries of Section 4.2 were trained separately, and Section 4.4 shows them doing less damage than the adversaries of this section on several variants. Third, a direct comparison of absolute PDR figures against the published FGSM numbers is not licensed, since the attacker here operates under a different constraint; that comparison belongs to the FGSM-parity clamp.

4.3.1 Independent scope: sensitivity to the timing budget

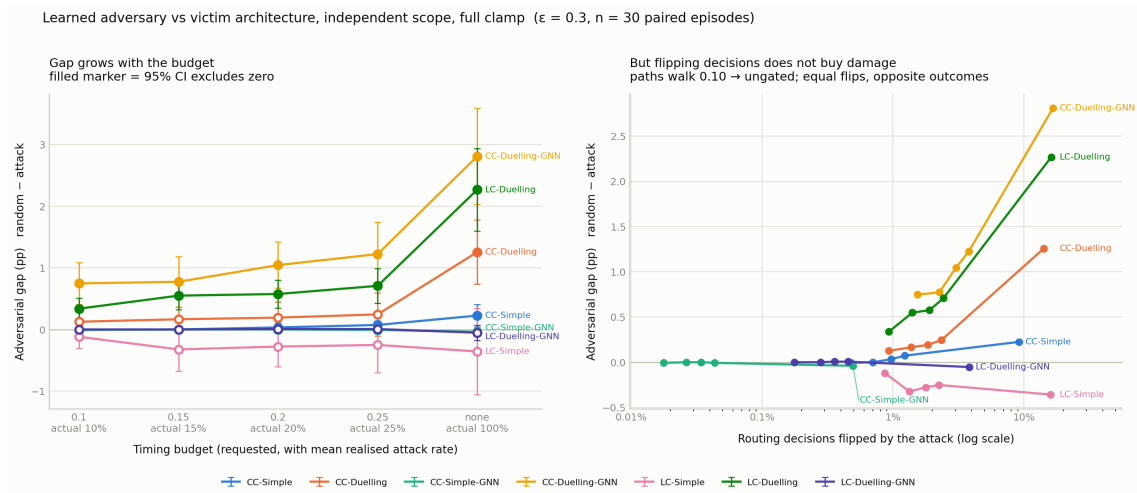


Figure 4.3: Independent scope, full domain clamp. Adversarial gap for all seven victim variants at $\varepsilon = 0.30$, $n = 30$ paired episodes. **Left:** gap against the requested timing budget, with the mean realised attack rate printed beneath each tick and the ungated run included as the 100% reference; filled markers denote a 95% confidence interval excluding zero, hollow markers a null. **Right:** the same gaps plotted against the fraction of routing decisions the attack actually changed, on a logarithmic axis, with each variant drawn as one path walking from $b = 0.10$ through to the ungated run.

The left panel of Figure 4.3 shows the gap rising monotonically with the timing budget for every variant that is attackable at all. The realised attack rates printed under the ticks confirm that the gate spends the budget it is given: requested budgets of 0.10, 0.15, 0.20 and 0.25 realise, averaged over the seven variants, as 10.0%, 15.0%, 19.9% and 24.9% respectively. This matters because the critical-state score has a point mass, which Section 4.2.3 locates at 0.90, so a large block of timesteps ties at exactly the trigger threshold at the tightest budgets; without the tie rationing described in Section 3.3.3 the realised rate cannot fall below the fraction of steps on that point mass, and the two tightest budgets collapse into nearly the same experiment.

Table 4.3 gives the underlying values. The architectural pattern is sharp: the duelling value head is the vulnerable axis. All three duelling variants without a local-critic GNN reach a significant gap at full rate, led by CC-Duelling-GNN at +2.81 pp and LC-Duelling at +2.27 pp, while every simple-head variant remains at or below +0.22 pp. Three variants, CC-Simple-GNN, LC-Simple and LC-Duelling-GNN, never separate from zero at any budget.

Table 4.3: Independent scope, full domain clamp. Adversarial gap in percentage points by victim variant and requested timing budget, at $\varepsilon = 0.30$ and $n = 30$ paired episodes. Bold entries have a 95% confidence interval excluding zero. Positive values indicate the attack cost the victim more than the random control of the same magnitude; negative values indicate the reverse.

Variant	0.10	0.15	0.20	0.25	ungated
CC-Simple	-0.01	+0.00	+0.03	+0.07	+0.22
CC-Duelling	+0.13	+0.17	+0.19	+0.24	+1.26
CC-Simple-GNN	-0.01	+0.00	+0.00	-0.01	-0.04
CC-Duelling-GNN	+0.75	+0.77	+1.04	+1.22	+2.81
LC-Simple	-0.12	-0.32	-0.28	-0.25	-0.36
LC-Duelling	+0.34	+0.55	+0.58	+0.71	+2.27
LC-Duelling-GNN	+0.00	+0.00	+0.01	+0.01	-0.05

Under this clamp, the budget sweep also supports the efficiency claim that motivated the timing gate. Normalising each gap by the realised attack rate gives the damage bought per attacked timestep, and for CC-Duelling-GNN this falls from 7.3 pp per unit rate at $b = 0.10$ to 2.8 pp when ungated, a factor of 2.6; for LC-Duelling the same ratio is 1.5. For these adversaries, concentrating the budget on high-utilisation states is therefore not merely cheaper but more efficient per step spent, which is the property that motivates dividing the attack across time in the manner of PGD. The effect is not universal: CC-Duelling shows essentially no efficiency gain, its damage scaling in proportion to the number of steps attacked, and under the FGSM-parity clamp no variant shows one (Section 4.2.1).

One caveat applies to the ungated column. Because the adversaries were trained with the gate at $b = 0.25$, evaluating them without a gate asks them to act on the remaining 75% of states they never encountered during training. That the ungated run is nonetheless the strongest point for every attackable variant indicates the learned perturbation direction generalises beyond the state distribution it was fitted on, rather than being specialised to

high-utilisation moments.

4.3.2 Independent scope: decision flips versus delivered damage

The right panel of Figure 4.3 addresses the assumption implicit in reporting flip rates as an attack outcome. Within any single variant the relationship holds in the expected direction: as the budget loosens, more routing decisions change and more damage follows, so each path slopes upward. Across variants the relationship fails completely. At the right-hand edge of the panel four variants sit within a narrow band of 14–17% of decisions flipped, yet their gaps span from -0.36 pp to $+2.81$ pp, a spread of more than three percentage points at essentially identical flip rates. Table 4.4 gives these figures directly.

Table 4.4: Independent scope, full domain clamp. Ungated results at $\varepsilon = 0.30$, $n = 30$ paired episodes: adversarial gap with its 95% confidence interval, and the fraction of routing decisions changed by the attack and by the random control.

Variant	Gap (pp)	95% CI	Attack flips	Random flips
CC-Simple	+0.22	[+0.04, +0.41]	9.15%	6.80%
CC-Duelling	+1.26	[+0.73, +1.78]	14.14%	12.02%
CC-Simple-GNN	-0.04	[-0.10, +0.02]	0.50%	0.34%
CC-Duelling-GNN	+2.81	[+2.03, +3.59]	16.69%	15.45%
LC-Simple	-0.36	[-1.06, +0.34]	15.94%	12.91%
LC-Duelling	+2.27	[+1.60, +2.94]	16.16%	11.98%
LC-Duelling-GNN	-0.05	[-0.18, +0.07]	3.82%	2.18%

LC-Simple is the clearest counterexample. Its path in the right panel runs downward as it moves right, so the more routing decisions the attack changes, the *better* the network performs relative to the random control. Flipping a routing decision is evidently not the same as flipping it to a worse path, and on this variant the learned direction is, on balance, no worse than noise. The random control is itself instructive here: it changes 12–15% of decisions on the high-flip variants, which means a raw flip count says almost nothing about whether an attacker is doing anything an untargeted disturbance would not.

The logarithmic axis separates two failure modes that a linear axis renders identical. CC-Simple-GNN never moves the argmax at all, flipping between 0.02% and 0.50% of decisions across the entire sweep; the perturbation is not being absorbed downstream, it is failing to change the selected path in the first place. This is the signature either of a policy whose decision boundaries lie beyond the ε -ball, or of an adversary that has failed to learn a perturbation able to cross them. Since the learned adversary never differentiates the victim, gradient masking in the usual sense cannot be the cause. LC-Duelling-GNN behaves quite differently: it flips 3.82% of decisions at full rate, more than seven times CC-Simple-GNN’s rate, and still returns a gap of -0.05 pp. Here the routing genuinely changes and the packets arrive regardless, which is absorption in the network rather than insensitivity in the policy. The distinction matters for what would follow: the first case might yield to a stronger optimiser or a larger budget, whereas the second reflects path

redundancy that no observation-space perturbation of this form is likely to overcome.

4.3.3 Coordinated scope

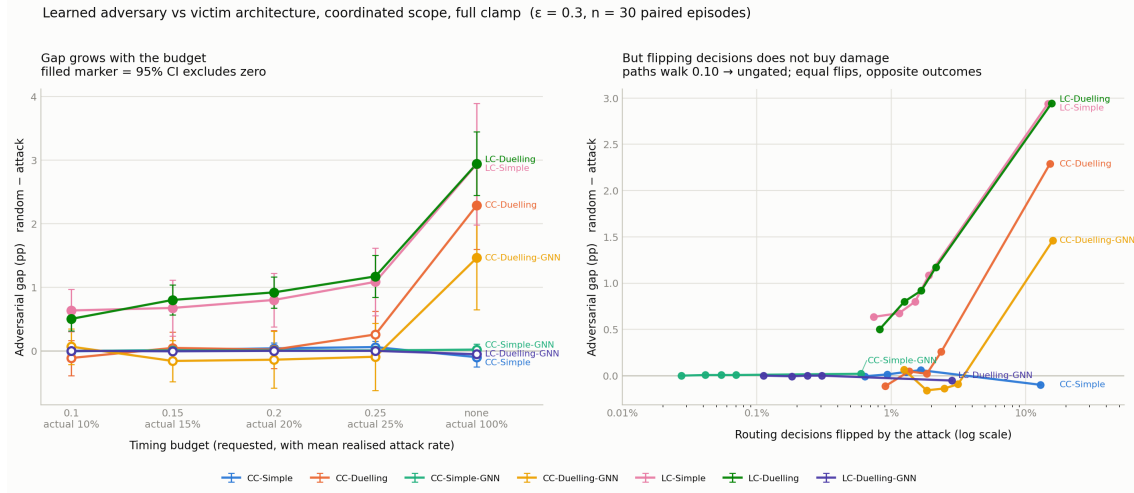


Figure 4.4: Coordinated scope, full domain clamp. Adversarial gap for all seven victim variants at $\varepsilon = 0.30$, $n = 30$ paired episodes, with the perturbation emitted jointly for the whole compromised group. Panels and markers as in Figure 4.3. A thin line joins a label to its series where the label had to be moved clear of the data.

Figure 4.4 repeats the sweep with the perturbation coordinated across the compromised agents, and Table 4.5 gives the values. The timing gate behaves identically in both scopes, realising 10.0%, 15.0%, 19.9% and 24.9% of steps at the four budgets, so any difference from the independent scope is due to the coordination of the perturbation rather than to how often it is applied.

The architectural pattern, however, changes markedly. The two local-critic variants without GNN encoding become the most vulnerable, and both are attacked significantly at *every* budget. LC-Simple, which the independent attack never harmed, rises monotonically from +0.63 pp at $b = 0.10$ to +1.08 pp at $b = 0.25$ and +2.94 pp ungated. LC-Duelling follows the same course, from +0.50 pp to +1.17 pp and +2.94 pp. The central-critic duelling variants, in contrast, are attacked significantly only when the adversary acts on every step: CC-Duelling reaches +2.29 pp and CC-Duelling-GNN +1.46 pp ungated, but neither separates from zero at any gated budget. CC-Simple, CC-Simple-GNN and LC-Duelling-GNN remain unaffected throughout.

Where coordination works, dividing the attack across time remains efficient. The damage bought per attacked step for LC-Simple is 6.3 pp per unit rate at $b = 0.10$ against 2.9 pp ungated, a factor of 2.2, and for LC-Duelling it is 5.0 pp against 2.9 pp, a factor of 1.7. The right panel repeats the lesson of Section 4.3.2: the paths of LC-Simple and LC-Duelling nearly coincide, ending at +2.94 pp with 14.7% and 15.6% of decisions flipped, while CC-Duelling-GNN ends at only +1.46 pp despite flipping 16.0%, more than either.

Table 4.5: Coordinated scope, full domain clamp. Adversarial gap in percentage points by victim variant and requested timing budget, at $\varepsilon = 0.30$ and $n = 30$ paired episodes. Bold entries have a 95% confidence interval excluding zero.

Variant	0.10	0.15	0.20	0.25	ungated
CC-Simple	-0.01	+0.01	+0.04	+0.06	-0.10
CC-Duelling	-0.11	+0.05	+0.02	+0.26	+2.29
CC-Simple-GNN	+0.00	+0.01	+0.01	+0.01	+0.02
CC-Duelling-GNN	+0.07	-0.16	-0.14	-0.09	+1.46
LC-Simple	+0.63	+0.67	+0.80	+1.08	+2.94
LC-Duelling	+0.50	+0.80	+0.92	+1.17	+2.94
LC-Duelling-GNN	+0.00	-0.01	+0.00	+0.00	-0.05

4.3.4 Effect of coordination

The two scopes can be compared exactly. Both were evaluated on the same traffic seed, and because neither the clean arm nor the random control depends on the adversary, their per-episode PDR series are bit-identical across the two scopes in all 35 configurations. The difference between the coordinated and independent adversarial gaps is then exactly the per-episode difference between the two attack arms,

$$\Delta^{\text{coord}} = \Delta_{\text{coord}}^{\text{adv}} - \Delta_{\text{ind}}^{\text{adv}} = \frac{1}{n} \sum_{e=1}^n \left(\text{PDR}_e^{\text{atk,ind}} - \text{PDR}_e^{\text{atk,coord}} \right), \quad (4.1)$$

which is paired within each episode and carries a confidence interval of the same form as Equation (3.20). A positive value means coordination increased the damage done. Table 4.6 reports it for every configuration.

Table 4.6: Effect of coordination under the full domain clamp: coordinated minus independent adversarial gap, in percentage points, by victim variant and requested timing budget ($\varepsilon = 0.30$, $n = 30$ paired episodes). Positive values mean coordination increased the damage. Bold entries have a 95% confidence interval on the paired difference of Equation (4.1) excluding zero.

Variant	0.10	0.15	0.20	0.25	ungated
CC-Simple	+0.00	+0.01	+0.01	-0.01	-0.32
CC-Duelling	-0.24	-0.12	-0.17	+0.01	+1.03
CC-Simple-GNN	+0.01	+0.01	+0.01	+0.01	+0.06
CC-Duelling-GNN	-0.68	-0.93	-1.18	-1.32	-1.35
LC-Simple	+0.75	+1.00	+1.08	+1.34	+3.29
LC-Duelling	+0.17	+0.25	+0.34	+0.46	+0.67
LC-Duelling-GNN	+0.00	-0.01	-0.01	-0.01	+0.00

Coordination does something, but not uniformly. On the two local-critic variants without GNN encoding it increases damage significantly at every budget. For LC-Simple the gain grows from +0.75 pp at $b = 0.10$ to +3.29 pp ungated, turning a variant the independent attack could not harm into one of the most damaged in the study. For LC-Duelling, already vulnerable, the gain grows from +0.17 pp to +0.67 pp. On the

central-critic variants the effect is mixed. It is nil at every gated budget except for CC-Duelling-GNN, where coordination significantly *reduces* damage at every budget, by between 0.68 and 1.35 pp. It appears only when attacking every step, where it helps CC-Duelling (+1.03 pp) and hurts CC-Simple (−0.32 pp). The two GNN variants that resisted the independent attack are unaffected by coordination as well.

The gains do not come from changing more decisions. On LC-Simple at $b = 0.25$ the coordinated attack flips fewer routing decisions than the independent one, 1.91% against 2.24%, while the gap moves from −0.25 pp to +1.08 pp; ungated, its flip rate falls from 15.94% to 14.65% as the gap rises by more than three percentage points. The converse also occurs: on CC-Simple coordination raises the ungated flip rate from 9.15% to 12.94% while the gap falls. What coordination changes is *which* decisions change together, and only that relates to delivered damage.

The concentration of the gains on the local-critic variants is consistent with how the two critic domains of Equation (3.1) are trained. A local critic conditions only on its own agent’s observation and action, so the policies it trains never receive a signal about how several agents’ routing choices combine on a shared link. A central critic conditions on the joint action, so its policies are trained against a value model that accounts for exactly those joint consequences. A perturbation that steers several agents onto the same link targets what the local critic cannot see. This explanation is consistent with the pattern but is not established by it: in particular, it does not account for why the effect disappears for LC-Duelling-GNN, and it does not predict the loss on CC-Duelling-GNN.

That loss calls for a specific caution. As described in Section 3.3.4, the coordinated adversary acts on the joint observation of all fourteen compromised agents, yet stores one replay transition per step rather than one per agent, so it learns from far fewer and far higher-dimensional samples than the independent adversary does. The large gains on the local-critic variants, obtained under the same condition, show that the reduced sample count alone does not explain the overall pattern. It cannot, however, be ruled out as the cause of the deficit on CC-Duelling-GNN in particular, and separating an under-trained adversary from a genuine effect of coordination on that variant would require an analysis of the coordinated adversary’s learning signal.

4.4 Effect of the Domain Clamp

The two clamps can be compared on paired episodes in the independent scope. The clean arm is bit-identical across the clamps, as noted in Section 4.2, but the random control is not, since each clamp’s control draws from its own admissible set. The difference between the two adversarial gaps is nevertheless paired within each episode,

$$\Delta^{\text{clamp}} = \Delta_{\text{parity}}^{\text{adv}} - \Delta_{\text{full}}^{\text{adv}} = \frac{1}{n} \sum_{e=1}^n \left[(\text{PDR}_e^{\text{rnd,parity}} - \text{PDR}_e^{\text{atk,parity}}) - (\text{PDR}_e^{\text{rnd,full}} - \text{PDR}_e^{\text{atk,full}}) \right], \quad (4.2)$$

and carries a confidence interval of the same form as Equation (3.20). A positive value means the parity-clamp adversary did more damage, relative to its own random control, than the full-clamp adversary. Table 4.7 reports it for every configuration.

Table 4.7: Effect of the domain clamp in the independent scope: FGSM-parity minus full clamp adversarial gap, in percentage points, by victim variant and requested timing budget ($\varepsilon = 0.30$, $n = 30$ paired episodes). Positive values mean the parity-clamp adversary did more damage. Bold entries have a 95% confidence interval on the paired difference of Equation (4.2) excluding zero.

Variant	0.10	0.15	0.20	0.25	ungated
CC-Simple	+0.00	+0.02	-0.05	-0.03	-0.24
CC-Duelling	-0.17	-0.07	-0.01	-0.16	-0.67
CC-Simple-GNN	+0.01	+0.00	-0.01	+0.01	+0.08
CC-Duelling-GNN	-0.92	-1.02	-0.93	-1.37	-2.03
LC-Simple	+0.52	+1.33	+1.55	+1.70	+6.26
LC-Duelling	-0.82	-1.26	-1.61	-1.53	-3.58
LC-Duelling-GNN	+0.00	+0.01	+0.00	-0.01	-0.13

The clamp changes the result significantly on three variants at every budget, and not in one direction. LC-Simple gains between 0.52 and 6.26 pp under the parity clamp, while CC-Duelling-GNN loses between 0.92 and 2.03 pp and LC-Duelling between 0.82 and 3.58 pp. These changes come from the attack arm rather than from the control. Ungated, the parity-clamp attack arm delivers 6.92 pp less than the full-clamp attack arm on LC-Simple, and 2.80 and 3.68 pp more on CC-Duelling-GNN and LC-Duelling, while the random control moves by less than one percentage point on all three. On CC-Duelling both arms move: ungated, the parity-clamp attack arm delivers 2.49 pp more, but its random control also delivers 1.82 pp more, and the net change of -0.67 pp is not significant. CC-Simple and CC-Simple-GNN change significantly only ungated, and by less than 0.25 pp, and LC-Duelling-GNN does not change.

Two conclusions follow. First, each measured gap describes a trained adversary, not a bound on what the victim can suffer. The parity set contains the full-clamp set, so an optimal adversary could do at least as much damage under it; the trained adversaries instead do less on two variants, and on LC-Duelling the sign reverses. A null result is correspondingly weak evidence of robustness: LC-Simple, which the full-clamp adversary does not harm at any budget, is the most damaged variant under the parity clamp, and is also harmed by the coordinated full-clamp adversary of Section 4.3.3. Second, the architectural conclusion of Section 4.3.1, that the duelling value head is the vulnerable axis, does not survive retraining the adversary under the baseline’s admissible set. The findings that hold under both clamps are narrower. CC-Simple-GNN and LC-Duelling-GNN are not harmed at any budget under either clamp, and CC-Simple by at most 0.22 pp; and under both clamps the fraction of decisions an attack changes does not predict the damage it does.

The two sets differ only outside the bandwidth block, where the parity clamp allows

features to be pushed below zero or above one, outside the $[0, 1]$ range that the full clamp enforces. Whether access to these out-of-range inputs is what makes LC-Simple vulnerable, and what misleads the adversary trained against LC-Duelling, has not been tested.

BIBLIOGRAPHY

- [1] The L^AT_EX Project team. *L^AT_EX - A document preparation system*. 1985. URL: <https://www.latex-project.org> (visited on 2025-12-29) (cit. on p. iii).
- [2] J. M. Lourenço. *The aidisclose package: Generative AI disclosure checklist and statements*. Version 1.6.2. 2025. URL: <https://ctan.org/pkg/aidisclose> (cit. on p. iii).
- [3] Y. Suchikova, N. Tsybuliak, J. A. T. da Silva, and S. Nazarovets. “GAIDeT (Generative AI Delegation Taxonomy): A Taxonomy for Humans to Delegate Tasks to Generative Artificial Intelligence in Scientific Research and Publishing”. In: *Accountability in Research* (2025), pp. 1–27. DOI: 10.1080/08989621.2025.2544331 (cit. on p. iii).
- [4] C. Tian, H. Cao, J. Xie, S. Garg, J. Rodrigues, and M. S. Hossain. “MADDPG-empowered slice reconfiguration approach for 5G multi-tier system”. In: *Journal of Network and Computer Applications* 219 (2023-09), p. 103725. DOI: 10.1016/j.jnca.2023.103725 (cit. on pp. 1, 9).
- [5] R. Lowe, Y. I. Wu, A. Tamar, J. Harb, O. Pieter Abbeel, and I. Mordatch. “Multi-agent actor-critic for mixed cooperative-competitive environments”. In: *Advances in neural information processing systems* 30 (2017) (cit. on pp. 1, 8, 27, 29, 34).
- [6] P. Almasan, J. Suárez-Varela, K. Rusek, P. Barlet-Ros, and A. Cabellos-Aparicio. “Deep reinforcement learning meets graph neural networks: Exploring a routing optimization use case”. In: *Computer Communications* 196 (2022-12), pp. 184–194. ISSN: 0140-3664. DOI: 10.1016/j.comcom.2022.09.029 (cit. on p. 1).
- [7] H. Wang, Y. Liu, W. Li, and Z. Yang. “Multi-agent deep reinforcement learning-based fine-grained traffic scheduling in data center networks”. In: *Future Internet* 16.4 (2024), p. 119 (cit. on p. 1).
- [8] W. Huang, H. Liu, Y. Li, and L. Ma. “ERA-MADDPG: An Elastic Routing Algorithm Based on Multi-Agent Deep Deterministic Policy Gradient in SDN”. In: *Future Internet* 17.7 (2025), p. 291 (cit. on pp. 1, 9).

- [9] S. S. Bhavanasi, L. Pappone, and F. Esposito. "Routing with Graph Convolutional Networks and Multi-Agent Deep Reinforcement Learning". In: *2022 IEEE Conference on Network Function Virtualization and Software Defined Networks (NFV-SDN)*. 2022, pp. 72–77. DOI: 10.1109/NFV-SDN56302.2022.9974607 (cit. on p. 1).
- [10] W. Zhang, C. Liu, Y. Pi, Y. Zhang, H. Huang, B. Rao, Y. Ding, S. Yang, and J. Jiang. *DRAMA: A Dynamic Packet Routing Algorithm using Multi-Agent Reinforcement Learning with Emergent Communication*. 2025. arXiv: 2504.04438 [cs.NI] (cit. on pp. 1, 9, 10).
- [11] F. Alanazi and M. Zareei. "Multi-Agent Deep Reinforcement Learning for Dynamic Routing in MANETs Using Graph Neural Networks". In: *IEEE Access* 13 (2025), pp. 152469–152478. DOI: 10.1109/ACCESS.2025.3601994 (cit. on p. 1).
- [12] C. Szegedy, W. Zaremba, I. Sutskever, J. Bruna, D. Erhan, I. Goodfellow, and R. Fergus. *Intriguing properties of neural networks*. 2014. arXiv: 1312.6199 [cs.CV] (cit. on pp. 1, 5).
- [13] I. J. Goodfellow, J. Shlens, and C. Szegedy. *Explaining and Harnessing Adversarial Examples*. 2015. arXiv: 1412.6572 [stat.ML] (cit. on pp. 1, 12, 15, 16, 28).
- [14] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, and A. Vladu. *Towards Deep Learning Models Resistant to Adversarial Attacks*. 2019. arXiv: 1706.06083 [stat.ML] (cit. on pp. 1, 2, 12, 16, 17, 27, 29, 34, 35).
- [15] N. Carlini and D. Wagner. *Towards Evaluating the Robustness of Neural Networks*. 2017. arXiv: 1608.04644 [cs.CR] (cit. on p. 1).
- [16] H. Zhang, H. Chen, C. Xiao, B. Li, M. Liu, D. Boning, and C.-J. Hsieh. "Robust Deep Reinforcement Learning against Adversarial Perturbations on State Observations". In: *Advances in Neural Information Processing Systems*. Vol. 33. 2020, pp. 21024–21037 (cit. on pp. 1, 3, 11, 12, 38).
- [17] P. Amaral and D. Raimundo. "Multi-Agent Deep Reinforcement Learning for Adaptive Routing: A Study of Critic Design and Graph Neural Network State Encoding". In: (2021) (cit. on pp. 2, 27–29).
- [18] W. Guo, G. Liu, and Z. Zhou. *SAJA: A State-Action Joint Attack Framework on Multi-Agent Deep Reinforcement Learning*. 2025. arXiv: 2510.13262 [cs.AI] (cit. on pp. 2, 12, 13, 27, 29, 32, 34).
- [19] T. Theodoropoulos, D. Kafetzis, J. Violos, A. Makris, and K. Tserpes. "Multi-agent deep reinforcement learning for weighted multi-path routing". In: *Proceedings of the 3rd Workshop on Flexible Resource and Application Management on the Edge*. 2023, pp. 7–11 (cit. on p. 5).
- [20] M. M. Alam and S. Moh. "Joint Trajectory Control, Frequency Allocation, and Routing for UAV Swarm Networks: A Multi-Agent Deep Reinforcement Learning Approach". In: *IEEE Transactions on Mobile Computing* 23.12 (2024), pp. 11989–12005. DOI: 10.1109/TMC.2024.3403890 (cit. on p. 5).

- [21] F. Lozano-Cuadra, M. D. Thorsager, I. Leyva-Mayorga, and B. Soret. *An open source Multi-Agent Deep Reinforcement Learning Routing Simulator for satellite networks*. 2024. arXiv: 2407.11047 [cs.LG] (cit. on p. 5).
- [22] P. Liu, X. Bai, and H. Cheng. "DRL-MR: Multipath Routing Based on Multi-Agent Deep Reinforcement Learning for SDN-Based Data Center Networks". In: *2025 International Joint Conference on Neural Networks (IJCNN)*. 2025, pp. 1–7. doi: 10.1109/IJCNN64981.2025.11227800 (cit. on p. 5).
- [23] R. A. Alliche, R. A. Pardo, and L. Sassatelli. "O-DQR: A Multi-Agent Deep Reinforcement Learning for Multihop Routing in Overlay Networks". In: *IEEE Transactions on Network and Service Management* 22.1 (2025), pp. 439–455. doi: 10.1109/TNSM.2024.3485196 (cit. on p. 5).
- [24] Z. Jia, S. He, Q. Zhu, W. Wang, Q. Wu, and Z. Han. "Trusted Routing for Blockchain-Empowered UAV Networks via Multi-Agent Deep Reinforcement Learning". In: *IEEE Transactions on Communications* 73.12 (2025), pp. 14227–14242. doi: 10.1109/TCOMM.2025.3597655 (cit. on p. 5).
- [25] Z. Zhou, G. Liu, W. Guo, and M. Zhou. "Adversarial Attacks on Multiagent Deep Reinforcement Learning Models in Continuous Action Space". In: *IEEE Transactions on Systems, Man, and Cybernetics: Systems* 54.12 (2024), pp. 7633–7646. doi: 10.1109/TSMC.2024.3454118 (cit. on pp. 5, 8).
- [26] Y.-C. Lin, Z.-W. Hong, Y.-H. Liao, M.-L. Shih, M.-Y. Liu, and M. Sun. *Tactics of Adversarial Attack on Deep Reinforcement Learning Agents*. 2019. arXiv: 1703.06748 [cs.LG] (cit. on p. 5).
- [27] S. Yamabe, K. Fukuchi, and J. Sakuma. *Robust Deep Reinforcement Learning against Adversarial Behavior Manipulation*. 2025. arXiv: 2406.03862 [cs.LG] (cit. on p. 5).
- [28] J. Dai and H. Sun. *Effective backdoor attack on graph neural networks in link prediction tasks*. 2025. arXiv: 2401.02663 [cs.LG] (cit. on pp. 5, 14, 15).
- [29] A. Alzubaidi, A. S. Al-Sumaiti, and M. Khonji. "Adversarial Deep Reinforcement Learning Attacks on Multi-Agent Autonomous Cooperative Driving Policies". In: *IET Intelligent Transport Systems* 19.1 (2025), e70066. doi: <https://doi.org/10.1049/itr2.70066> (cit. on p. 5).
- [30] Z. Zhou, G. Liu, W. Guo, and M. Zhou. "Adversarial Attacks on Multiagent Deep Reinforcement Learning Models in Continuous Action Space". In: *IEEE Transactions on Systems, Man, and Cybernetics: Systems* 54.12 (2024), pp. 7633–7646. doi: 10.1109/TSMC.2024.3454118 (cit. on p. 5).
- [31] S. G. Finlayson, J. D. Bowers, J. Ito, J. L. Zittrain, A. L. Beam, and I. S. Kohane. "Adversarial attacks on medical machine learning". In: *Science* 363.6433 (2019), pp. 1287–1289. doi: 10.1126/science.aaw4399 (cit. on p. 5).

- [32] W. Villegas-Ch, A. Jaramillo-Alcázar, and S. Luján-Mora. "Evaluating the robustness of deep learning models against adversarial attacks: An analysis with fgsm, pgd and cw". In: *Big Data and Cognitive Computing* 8.1 (2024), p. 8 (cit. on pp. 6, 16).
- [33] T. Duan, Z. Zhang, Z. Lin, Y. Gao, L. Xiong, Y. Cui, H. Liang, X. Chen, H. Cui, and D. Huang. "Rethinking Adversarial Attacks in Reinforcement Learning from Policy Distribution Perspective". In: *ICASSP 2025 - 2025 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. 2025, pp. 1–5. DOI: 10.1109/ICASSP49660.2025.10890540 (cit. on p. 6).
- [34] V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, and M. Riedmiller. *Playing Atari with Deep Reinforcement Learning*. 2013. arXiv: 1312.5602 [cs.LG] (cit. on pp. 6, 7).
- [35] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, et al. "Human-level control through deep reinforcement learning". In: *nature* 518.7540 (2015), pp. 529–533 (cit. on p. 6).
- [36] X. Geng and B. Zhang. "Deep Q-Network-Based Intelligent Routing Protocol for Underwater Acoustic Sensor Network". In: *IEEE Sensors Journal* 23.4 (2023), pp. 3936–3943. DOI: 10.1109/JSEN.2023.3234112 (cit. on p. 7).
- [37] Q. Fu, E. Sun, K. Meng, M. Li, and Y. Zhang. "Deep Q-Learning for Routing Schemes in SDN-Based Data Center Networks". In: *IEEE Access* 8 (2020), pp. 103491–103499. DOI: 10.1109/ACCESS.2020.2995511 (cit. on p. 7).
- [38] Z. Wang, T. Schaul, M. Hessel, H. van Hasselt, M. Lanctot, and N. de Freitas. *Dueling Network Architectures for Deep Reinforcement Learning*. 2016. arXiv: 1511.06581 [cs.LG] (cit. on pp. 7, 8).
- [39] M. Zhao. *Breaking the Cold-Start Barrier: Reinforcement Learning with Double and Dueling DQNs*. 2025. arXiv: 2508.21259 [cs.IR] (cit. on p. 7).
- [40] J. Zhang, F. Qian, and J. Yang. "Online routing and spectrum allocation in elastic optical networks based on dueling Deep Q-network". In: *Computers & Industrial Engineering* 173 (2022), p. 108663. ISSN: 0360-8352. DOI: <https://doi.org/10.1016/j.cie.2022.108663> (cit. on p. 8).
- [41] N. H. Pham, L. M. Nguyen, J. Chen, H. T. Lam, S. Das, and L. Weng. "c-MBA: Adversarial attack for cooperative MARL using learned dynamics model". In: *NeurIPS ML Safety Workshop*. 2022 (cit. on p. 8).
- [42] S. Li, J. Guo, J. Xiu, Y. Zheng, P. Feng, X. Yu, J. Wang, A. Liu, Y. Yang, B. An, et al. "Attacking cooperative multi-agent reinforcement learning by adversarial minority influence". In: *Neural Networks* (2025), p. 107747 (cit. on pp. 8, 14, 15).
- [43] S. Kuang, J. Zheng, S. Liang, Y. Li, S. Liang, and W. Huang. "RS-MADDPG: Routing Strategy Based on Multi-Agent Deep Deterministic Policy Gradient for Differentiated QoS Services". In: *Future Internet* 17.9 (2025), p. 393 (cit. on p. 9).

-
- [44] H. Zhang, H. Chen, D. Boning, and C.-J. Hsieh. “Robust Reinforcement Learning on State Observations with Learned Optimal Adversary”. In: *International Conference on Learning Representations*. 2021 (cit. on p. 12).
 - [45] Y. Sun, R. Zheng, Y. Liang, and F. Huang. “Who is the strongest enemy? towards optimal and efficient evasion attacks in deep rl”. In: *arXiv preprint arXiv:2106.05087* (2021) (cit. on pp. 12, 13).
 - [46] H. Waghela, J. Sen, and S. Rakshit. “Robust Image Classification: Defensive Strategies against FGSM and PGD Adversarial Attacks”. In: *2024 Asian Conference on Intelligent Technologies (ACOIT)*. IEEE, 2024-09, pp. 1–7. doi: 10.1109/acoit62457.2024.10941671 (cit. on p. 12).
 - [47] S. Lee, J. Hwang, Y. Jo, and S. Han. *Wolfpack Adversarial Attack for Robust Multi-Agent Reinforcement Learning*. 2025. arXiv: 2502.02844 [cs.LG] (cit. on p. 14).
 - [48] Z. Wu, S. C. Ye, B. Han, and M. Gombolay. “Hijacking Robot Teams Through Adversarial Communication”. In: *7th Annual Conference on Robot Learning*. 2023 (cit. on p. 14).
 - [49] C. Zhao, Y. Ze, J. Dong, B. Wang, and S. Li. “DPMAC: Differentially private communication for cooperative multi-agent reinforcement learning”. In: *arXiv preprint arXiv:2308.09902* (2023) (cit. on p. 14).
 - [50] L. Yuan, F. Chen, Z. Zhang, and Y. Yu. *Communication-Robust Multi-Agent Learning by Adaptable Auxiliary Multi-Agent Adversary Generation*. 2023. arXiv: 2305.05116 [cs.LG] (cit. on p. 14).
 - [51] W. Shen, H. Li, and Z. Zheng. “Coordinated attacks against federated learning: A multi-agent reinforcement learning approach”. In: *ICLR 2021 Workshop on Security and Safety in Machine Learning Systems (SecML)*. ICLR 2021 Workshop on Security and Safety in Machine Learning Systems (SecML). 2021 (cit. on p. 14).
 - [52] W. Guo, G. Liu, Z. Zhou, and L. Wang. “PNAct: Crafting Backdoor Attacks in Safe Reinforcement Learning”. In: *arXiv preprint arXiv:2507.00485* (2025) (cit. on p. 15).
 - [53] J. Guo, A. Li, L. Wang, and C. Liu. “Polycycleanse: Backdoor detection and mitigation for competitive reinforcement learning”. In: *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 2023, pp. 4699–4708 (cit. on p. 15).
 - [54] S. Chen, Y. Qiu, and J. Zhang. *Backdoor Attacks on Multiagent Collaborative Systems*. 2022. arXiv: 2211.11455 [cs.MA] (cit. on p. 15).
 - [55] X. Pan, Y. Cao, X. Wu, E. Zelikman, C. Xiao, Y. Sui, R. Chakraborty, and R. S. Fearing. “Transferable adversarial attacks on deep reinforcement learning with domain randomization”. In: (2020) (cit. on p. 15).
 - [56] N. Akhtar, A. Mian, N. Kardan, and M. Shah. “Advances in adversarial attacks and defenses in computer vision: A survey”. In: *IEEE access* 9 (2021), pp. 155161–155196 (cit. on pp. 16, 17).

- [57] A. Rozsa, E. M. Rudd, and T. E. Boulton. "Adversarial diversity and hard positive generation". In: *Proceedings of the IEEE conference on computer vision and pattern recognition workshops*. 2016, pp. 25–32 (cit. on p. 16).
- [58] A. Kurakin, I. Goodfellow, and S. Bengio. *Adversarial examples in the physical world*. 2016. arXiv: 1607.02533 [cs.CV] (cit. on pp. 16, 35).
- [59] A. Kurakin, I. Goodfellow, and S. Bengio. *Adversarial Machine Learning at Scale*. 2017. arXiv: 1611.01236 [cs.CV] (cit. on pp. 16, 17).
- [60] C. Xie, Z. Zhang, Y. Zhou, S. Bai, J. Wang, Z. Ren, and A. L. Yuille. "Improving Transferability of Adversarial Examples With Input Diversity". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2019-06 (cit. on p. 16).
- [61] J. Zhou, G. Cui, S. Hu, Z. Zhang, C. Yang, Z. Liu, L. Wang, C. Li, and M. Sun. *Graph Neural Networks: A Review of Methods and Applications*. 2021. arXiv: 1812.08434 [cs.LG] (cit. on pp. 19–21).
- [62] Z. Zhang, P. Cui, and W. Zhu. "Deep Learning on Graphs: A Survey". In: *IEEE Transactions on Knowledge and Data Engineering* 34.1 (2022), pp. 249–270. doi: 10.1109/TKDE.2020.2981333 (cit. on pp. 19–21).
- [63] Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, and P. S. Yu. "A Comprehensive Survey on Graph Neural Networks". In: *IEEE Transactions on Neural Networks and Learning Systems* 32.1 (2021), pp. 4–24. doi: 10.1109/TNNLS.2020.2978386 (cit. on pp. 19–21).
- [64] I. Chami, S. Abu-El-Haija, B. Perozzi, C. Ré, and K. Murphy. "Machine learning on graphs: A model and comprehensive taxonomy". In: *Journal of Machine Learning Research* 23.89 (2022), pp. 1–64 (cit. on pp. 19–21).
- [65] O. Rippel, J. Snoek, and R. P. Adams. "Spectral Representations for Convolutional Neural Networks". In: *Advances in Neural Information Processing Systems*. Ed. by C. Cortes, N. Lawrence, D. Lee, M. Sugiyama, and R. Garnett. Vol. 28. Curran Associates, Inc., 2015. URL: https://proceedings.neurips.cc/paper_files/paper/2015/file/536a76f94cf7535158f66cfbd4b113b6-Paper.pdf (cit. on p. 20).
- [66] S. Tang, B. Li, and H. Yu. "ChebNet: efficient and stable constructions of deep neural networks with rectified power units via Chebyshev approximation". In: *Communications in Mathematics and Statistics* (2024), pp. 1–27 (cit. on p. 20).
- [67] E. Jiawei, Y. Zhang, S. Yang, H. Wang, X. Xia, and X. Xu. "GraphSAGE++: Weighted multi-scale GNN for graph representation learning". In: *Neural Processing Letters* 56.1 (2024), p. 24 (cit. on p. 20).
- [68] M. W. Raja. "Artificial intelligence-based healthcare data analysis using multi-perceptron neural network (MPNN) based on optimal feature selection". In: *SN Computer Science* 5.8 (2024), p. 1034 (cit. on p. 20).

-
- [69] Z. Xu, L. Bai, L. Cui, M. Li, Y. Wang, and E. R. Hancock. "HC-GAE: The Hierarchical Cluster-based Graph Auto-Encoder for Graph Representation Learning". In: *arXiv preprint arXiv:2405.14742* (2024) (cit. on p. 20).
 - [70] M. Gao, S. Liu, Y. Qi, X. Guo, and X. Shang. "GAE-LGA: integration of multi-omics data with graph autoencoders to identify lncRNA-PCG associations". In: *Briefings in Bioinformatics* 23.6 (2022) (cit. on p. 20).
 - [71] G. Jin, M. Wang, J. Zhang, H. Sha, and J. Huang. "STGNN-TTE: Travel time estimation via spatial-temporal graph neural network". In: *Future Generation Computer Systems* 126 (2022), pp. 70–81 (cit. on p. 20).
 - [72] J. Yin, A. Qiu, L. Fang, N. Wang, C. Dong, and S. Ge. "STGNN: A Novel Spatial-Temporal Graph Neural Network for Predicting Complicated Business Process Performance under Multi-Event Parallelism". In: *Expert Systems with Applications* (2025), p. 128391 (cit. on p. 20).
 - [73] W. Jiang. "Graph-based deep learning for communication networks: A survey". In: *Computer Communications* 185 (2021-03), pp. 40–54. ISSN: 0140-3664. DOI: 10.1016/j.comcom.2021.12.015 (cit. on p. 21).
 - [74] K. Rusek, J. Suárez-Varela, A. Mestres, P. Barlet-Ros, and A. Cabellos-Aparicio. "Unveiling the potential of Graph Neural Networks for network modeling and optimization in SDN". In: *Proceedings of the 2019 ACM Symposium on SDN Research. SOSR '19*. San Jose, CA, USA: Association for Computing Machinery, 2019, pp. 140–151. ISBN: 9781450367103. DOI: 10.1145/3314148.3314357 (cit. on p. 21).
 - [75] H. Wei, Y. Zhao, and K. Xu. "G-Routing: graph neural networks-based flexible online routing". In: *IEEE Network* 37.4 (2023), pp. 90–96 (cit. on p. 21).
 - [76] M. Ding, Y. Guo, Z. Huang, B. Lin, and H. Luo. "GROM: A generalized routing optimization method with graph neural network and deep reinforcement learning". In: *J. Netw. Comput. Appl.* 229.C (2024-09). ISSN: 1084-8045. DOI: 10.1016/j.jnca.2024.103927 (cit. on p. 21).
 - [77] F. Zhou, X. Liu, R. Yu, and G. Xue. *Traffic Engineering in Large-scale Networks with Generalizable Graph Neural Networks*. 2025. arXiv: 2503.24203 [cs.NI] (cit. on pp. 21, 22).
 - [78] W. Jiang, H. Han, Y. Zhang, J. Wang, M. He, W. Gu, J. Mu, and X. Cheng. "Graph neural networks for routing optimization: Challenges and opportunities". In: *Sustainability* 16.21 (2024), p. 9239 (cit. on p. 22).
 - [79] L. Luo, W. Ren, H. Huang, and F. Wang. "A Survey on Privacy Attacks and Defenses in Graph Neural Networks". In: *Information Technology and Control* 53.4 (2024), pp. 1251–1278 (cit. on pp. 23–25).

- [80] J. Wei, L. Yaxin, X. Han, W. Yiqi, J. Shuiwang, and A. Charu. “Adversarial attacks and defenses on graphs: A review, a tool, and empirical studies”. In: *arXiv preprint arXiv:2003.00653* 653 (2020) (cit. on pp. 23–25).
- [81] D. Galli, A. Venturi, D. Stabili, M. Andreolini, and M. Marchetti. “Defending Network Intrusion Detection Systems Based on Graph Neural Networks Against Structural Adversarial Attacks”. In: *2025 23rd International Symposium on Network Computing and Applications (NCA)*. 2025, pp. 219–228. doi: 10.1109/NCA67271.2025.00043 (cit. on pp. 23, 24).
- [82] D. Zügner, A. Akbarnejad, and S. Günnemann. “Adversarial Attacks on Neural Networks for Graph Data”. In: *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. KDD ’18. London, United Kingdom: Association for Computing Machinery, 2018, pp. 2847–2856. ISBN: 9781450355520. doi: 10.1145/3219819.3220078 (cit. on p. 24).
- [83] M. Aburidi and R. Marcia. “Topological adversarial attacks on graph neural networks via projected meta learning”. In: (2024) (cit. on pp. 24, 25).
- [84] J. Chen, Y. Wu, X. Xu, Y. Chen, H. Zheng, and Q. Xuan. “Fast gradient attack on network embedding”. In: *arXiv preprint arXiv:1809.02797* (2018) (cit. on p. 24).
- [85] H. Dai, H. Li, T. Tian, X. Huang, L. Wang, J. Zhu, and L. Song. “Adversarial Attack on Graph Structured Data”. In: *Proceedings of the 35th International Conference on Machine Learning*. Ed. by J. Dy and A. Krause. Vol. 80. Proceedings of Machine Learning Research. PMLR, 2018-07, pp. 1115–1124. URL: <https://proceedings.mlr.press/v80/dai18b.html> (cit. on p. 24).
- [86] K. Xu, H. Chen, S. Liu, P.-Y. Chen, T.-W. Weng, M. Hong, and X. Lin. “Topology attack and defense for graph neural networks: An optimization perspective”. In: *arXiv preprint arXiv:1906.04214* (2019) (cit. on p. 24).
- [87] Y. Sun, S. Wang, X. Tang, T.-Y. Hsieh, and V. Honavar. “Adversarial Attacks on Graph Neural Networks via Node Injections: A Hierarchical Reinforcement Learning Approach”. In: *Proceedings of The Web Conference 2020*. WWW ’20. Taipei, Taiwan: Association for Computing Machinery, 2020, pp. 673–683. ISBN: 9781450370233. doi: 10.1145/3366423.3380149 (cit. on p. 24).
- [88] Y. Ma, S. Wang, T. Derr, L. Wu, and J. Tang. “Attacking graph convolutional networks via rewiring”. In: *arXiv preprint arXiv:1906.03750* (2019) (cit. on p. 24).
- [89] H. Wu, C. Wang, Y. Tyshetskiy, A. Docherty, K. Lu, and L. Zhu. “Adversarial examples on graph data: Deep insights into attack and defense”. In: *arXiv preprint arXiv:1903.01610* (2019) (cit. on p. 24).
- [90] M. Aburidi and R. Marcia. “Defending Graph Neural Networks Against Adversarial attacks via Symmetric Matrix Factorization”. In: *2025 Conference on Artificial Intelligence x Multimedia (AIXMM)*. 2025, pp. 55–61. doi: 10.1109/AIXMM62960.2025.00016 (cit. on p. 25).

- [91] L. Roy, P. Bhandari, S. Yadav, A. Akula, A. S. Reddy, and A. Lathigara. “Protecting Against Adversarial Attacks in Graph Neural Networks: Vulnerability Assessment and Parallel Defense Strategies”. In: *2025 World Skills Conference on Universal Data Analytics and Sciences (WorldSUAS)*. 2025, pp. 1–6. DOI: [10.1109/WorldSUAS66815.2025.11199215](https://doi.org/10.1109/WorldSUAS66815.2025.11199215) (cit. on p. 25).



2026

A Very Long and Impressive Thesis Title With a Forced Line Break Miguel Chen